

Module 8: Machine Learning I

Learning Objectives

- Understand fundamental machine learning concepts and terminology
- Learn about different types of machine learning problems
- Master data preprocessing and feature engineering techniques
- Implement train-test split and cross-validation strategies
- Get familiar with scikit-learn library
- Build and evaluate linear regression models
- Build and evaluate logistic regression models
- Understand decision tree algorithms
- Learn comprehensive model evaluation metrics
- Recognize and handle overfitting and underfitting
- Apply various feature selection techniques

Topics Covered

1. Introduction to machine learning
2. Types of machine learning problems
3. Data preprocessing and feature engineering
4. Train-test split and cross-validation
5. Introduction to scikit-learn
6. Linear regression
7. Logistic regression
8. Decision trees
9. Model evaluation metrics
10. Overfitting and underfitting
11. Feature selection techniques

Prerequisites

- Basic Python programming
- NumPy and Pandas
- Matplotlib for visualization
- Understanding of statistics concepts

Exercises

- Hands-on implementation of ML algorithms
- Model evaluation and validation exercises
- Feature engineering challenges

- Overfitting detection and prevention

1. Introduction to Machine Learning

In [...

```

1      # Import all necessary libraries
2      import numpy as np
3      import pandas as pd
4      import matplotlib.pyplot as plt
5      import seaborn as sns
6      from sklearn.model_selection import train_test_split, cross_val_s
7      from sklearn.linear_model import LinearRegression, LogisticRegres
8      from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegr
9      from sklearn.ensemble import RandomForestClassifier
10     from sklearn.metrics import (accuracy_score, mean_squared_error,
11                                  r2_score, classification_report, confu
12                                  precision_score, recall_score, f1_scor
13
14     from sklearn.preprocessing import StandardScaler, MinMaxScaler, L
15     from sklearn.impute import SimpleImputer
16     from sklearn.feature_selection import SelectKBest, f_classif, RFE
17     from sklearn.pipeline import Pipeline
18     from sklearn.compose import ColumnTransformer
19     from sklearn.datasets import make_classification, make_regression
20     import warnings
21     warnings.filterwarnings('ignore')
22
23     # Set style for better plots
24     plt.style.use('seaborn-v0_8')
25     sns.set_palette("husl")
26
27     print("All libraries imported successfully!")
28     print("Ready to start Machine Learning!")
29
30
31     # =====
32     # 1. INTRODUCTION TO MACHINE LEARNING
33     # =====
34
35     print("MACHINE LEARNING TYPES")
36     print("=" * 50)
37
38     print("\n1. SUPERVISED LEARNING")
39     print("    • Uses labeled training data (input-output pairs)")
40     print("    • Goal: Learn mapping from inputs to outputs")
41     print("    • Examples:")
42     print("        - Email spam detection (spam/not spam)")
43     print("        - House price prediction")
44     print("        - Medical diagnosis")
45     print("        - Image classification")
46
47
48     print("\n2. UNSUPERVISED LEARNING")
49     print("    • Uses unlabeled data")
50     print("    • Goal: Find hidden patterns in data")
51     print("    • Examples:")
52     print("        - Customer segmentation")
53     print("        - Anomaly detection")
54     print("        - Market basket analysis")
55     print("        - Dimensionality reduction")
56

```

```

57
58 print("\n3. REINFORCEMENT LEARNING")
59 print("    • Learns through interaction with environment")
60 print("    • Goal: Learn optimal actions through rewards")
61 print("    • Examples:")
62 print("        - Game playing (AlphaGo, Chess)")
63 print("        - Autonomous vehicles")
64 print("        - Trading algorithms")
65 print("        - Robotics")
66
67
68 print("\nCOMMON ML PROBLEMS")
69 print("=" * 30)
70 print("Classification: Predicting categories")
71 print("    Example: Spam detection, image recognition")
72 print("\nRegression: Predicting continuous values")
73 print("    Example: House prices, stock prices")
74 print("\nClustering: Grouping similar data points")
75 print("    Example: Customer segmentation, gene sequencing")
76 print("\nDimensionality Reduction: Reducing features")
77 print("    Example: PCA, t-SNE for visualization")
78
79
80 print("\nTYPICAL ML WORKFLOW")
81 print("=" * 25)
82 print("1. Data Collection")
83 print("2. Data Preprocessing")
84 print("3. Feature Engineering")
85 print("4. Model Selection")
86 print("5. Training")
87 print("6. Evaluation")
88 print("7. Deployment")
89 print("8. Monitoring & Maintenance")
90
91 # Create a visual representation
92 fig, ax = plt.subplots(1, 1, figsize=(12, 8))
93
94 # ML Types visualization
95 ml_types = ['Supervised', 'Unsupervised', 'Reinforcement']
96 examples = ['Classification\nRegression', 'Clustering\nDimensionality Reduction']
97
98 colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']
99 y_pos = [0.7, 0.5, 0.3]
100
101
102 for i, (ml_type, example, color) in enumerate(zip(ml_types, examples, colors)):
103     ax.text(0.1, y_pos[i], f'{ml_type} Learning', fontsize=16, fontweight='bold')
104     ax.text(0.1, y_pos[i]-0.05, example, fontsize=12, color=color)
105     ax.barh(y_pos[i], 0.8, height=0.08, color=color, alpha=0.3)
106
107 ax.set_xlim(0, 1)
108 ax.set_ylim(0, 1)
109 ax.set_title('Machine Learning Types', fontsize=18, fontweight='bold')
110 ax.axis('off')
111
112 plt.tight_layout()
113 plt.show()
114
115 print("\nKey Takeaway: Machine Learning is about teaching computers to learn from data")

```

2. Types of Machine Learning Problems

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61

```

```

# =====
# 2. TYPES OF MACHINE LEARNING PROBLEMS
# =====

print("MACHINE LEARNING PROBLEM TYPES")
print("=" * 40)

# 1. CLASSIFICATION PROBLEMS
print("\n1. CLASSIFICATION")
print("    Goal: Predict categorical labels")
print("    Output: Discrete classes/categories")
print("    Examples:")
print("        • Binary: Spam/Not Spam, Sick/Healthy")
print("        • Multiclass: Image recognition (Cat/Dog/Bird)")
print("        • Multilabel: Multiple tags per image")

# Generate classification data example
X_class, y_class = make_classification(n_samples=200, n_features=
                                     n_informative=2, n_clusters
                                     random_state=42)

# Visualize classification data
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Classification scatter plot
scatter = axes[0].scatter(X_class[:, 0], X_class[:, 1], c=y_class)
axes[0].set_title('Classification Problem\n(Two Classes)', fontsi
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')
plt.colorbar(scatter, ax=axes[0])

# 2. REGRESSION PROBLEMS
print("\n2. REGRESSION")
print("    Goal: Predict continuous numerical values")
print("    Output: Real numbers")
print("    Examples:")
print("        • House price prediction")
print("        • Stock price forecasting")
print("        • Temperature prediction")
print("        • Sales forecasting")

# Generate regression data example
X_reg, y_reg = make_regression(n_samples=200, n_features=1, noise

# Regression scatter plot
axes[1].scatter(X_reg, y_reg, alpha=0.7, color='orange')
axes[1].set_title('Regression Problem\n(Continuous Output)', font
axes[1].set_xlabel('Feature')
axes[1].set_ylabel('Target Value')

plt.tight_layout()
plt.show()

# 3. CLUSTERING PROBLEMS
print("\n3. CLUSTERING (UNSUPERVISED)")
print("    Goal: Group similar data points")
print("    Output: Cluster assignments")

```

```

62 print("    Examples:")
63 print("    • Customer segmentation")
64 print("    • Gene sequencing")
65 print("    • Market research")
66 print("    • Anomaly detection")
67
68 # Generate clustering data
69 from sklearn.datasets import make_blobs
70 X_cluster, y_cluster = make_blobs(n_samples=200, centers=3, n_fea
71                                   random_state=42, cluster_std=2)
72
73 # Visualize clustering data
74 plt.figure(figsize=(8, 6))
75 plt.scatter(X_cluster[:, 0], X_cluster[:, 1], c=y_cluster, cmap='
76 plt.title('Clustering Problem\n(Group Similar Data Points)', font
77 plt.xlabel('Feature 1')
78 plt.ylabel('Feature 2')
79 plt.show()
80
81 # 4. DIMENSIONALITY REDUCTION
82 print("\n4. DIMENSIONALITY REDUCTION")
83 print("    Goal: Reduce number of features while preserving inform
84 print("    Output: Lower-dimensional representation")
85 print("    Examples:")
86 print("    • Data visualization (2D/3D plots)")
87 print("    • Noise reduction")
88 print("    • Feature compression")
89 print("    • Data preprocessing")
90
91 # Demonstrate dimensionality reduction
92 from sklearn.decomposition import PCA
93 from sklearn.datasets import load_breast_cancer
94
95 # Load high-dimensional data
96 cancer = load_breast_cancer()
97 X_high_dim = cancer.data
98 y_high_dim = cancer.target
99
100 # Apply PCA
101 pca = PCA(n_components=2)
102 X_pca = pca.fit_transform(X_high_dim)
103
104 # Visualize dimensionality reduction
105 plt.figure(figsize=(12, 5))
106
107 plt.subplot(1, 2, 1)
108 plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_high_dim, cmap='viridis
109 plt.title('Dimensionality Reduction\n(30D → 2D using PCA)', fonts
110 plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance
111 plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance
112
113 plt.subplot(1, 2, 2)
114 plt.bar(range(1, 11), pca.explained_variance_ratio_[1:10])
115 plt.title('Explained Variance by Component', fontsize=14, fontwei
116 plt.xlabel('Principal Component')
117 plt.ylabel('Explained Variance Ratio')
118
119 plt.tight_layout()
120 plt.show()

```

```

125 print(f"\nPROBLEM TYPE SUMMARY")
126 print("=" * 25)
print(f"Classification: {len(np.unique(y_class))} classes, {X_cla
print(f"Regression: Continuous target, {X_reg.shape[1]} features"
print(f"Clustering: {len(np.unique(y_cluster))} clusters, {X_clus
print(f"Dimensionality Reduction: {X_high_dim.shape[1]}D → 2D")

print("\nKey Takeaway: Choose the right problem type based on you

```

3. Data Preprocessing and Feature Engineering

In [...

```

1      # =====
2      # 3. DATA PREPROCESSING AND FEATURE ENGINEERING
3      # =====
4
5
6      print("DATA PREPROCESSING & FEATURE ENGINEERING")
7      print("=" * 50)
8
9      print("\nGOALS:")
10     print("• Clean and prepare data for ML algorithms")
11     print("• Handle missing values, outliers, and inconsistencies")
12     print("• Transform categorical variables")
13     print("• Scale numerical features")
14     print("• Create new meaningful features")
15     print("• Select the most relevant features")
16
17     # Create a comprehensive dataset with various data types and issues
18     np.random.seed(42)
19     n_samples = 1000
20
21     print("\nCREATING SAMPLE DATASET")
22     print("-" * 30)
23
24     # Generate synthetic data with realistic patterns
25     data = {
26         'age': np.random.normal(35, 10, n_samples),
27         'income': np.random.normal(50000, 15000, n_samples),
28         'education': np.random.choice(['High School', 'Bachelor', 'Master', 'PhD'], n_samples),
29         'experience': np.random.randint(0, 30, n_samples),
30         'city': np.random.choice(['New York', 'Los Angeles', 'Chicago', 'San Francisco', 'London'], n_samples),
31         'department': np.random.choice(['IT', 'HR', 'Finance', 'Marketing', 'Operations'], n_samples),
32         'performance_score': np.random.normal(75, 15, n_samples),
33         'satisfaction': np.random.choice(['Low', 'Medium', 'High'], n_samples),
34         'salary': np.random.normal(60000, 20000, n_samples),
35         'years_at_company': np.random.randint(0, 15, n_samples)
36     }
37
38     # Create DataFrame
39     df = pd.DataFrame(data)
40
41     # Introduce realistic missing values (some patterns)
42     missing_indices = np.random.choice(df.index, size=int(0.1 * len(df)), replace=True)
43     df.loc[missing_indices, 'income'] = np.nan
44
45     missing_indices = np.random.choice(df.index, size=int(0.05 * len(df)), replace=True)
46     df.loc[missing_indices, 'education'] = np.nan
47
48

```

```

49
50 # Introduce outliers (high earners)
51 outlier_indices = np.random.choice(df.index, size=int(0.02 * len(
52 df.loc[outlier_indices, 'salary'] = df.loc[outlier_indices, 'sala
53
54 print(f"Dataset created: {df.shape[0]} rows, {df.shape[1]} column
55 print(f"Features: {list(df.columns)}")
56
57 print("\nDATASET OVERVIEW")
58 print("-" * 20)
59 print(f"\nFirst 5 rows:")
60 print(df.head())
61
62
63 print(f"\nData types:")
64 print(df.dtypes)
65
66 print(f"\nMissing values:")
67 missing_summary = df.isnull().sum()
68 print(missing_summary[missing_summary > 0])
69
70 print(f"\nBasic statistics:")
71 print(df.describe())
72
73 # 1. HANDLING MISSING VALUES
74 print("\n1. HANDLING MISSING VALUES")
75 print("-" * 35)
76
77 # Check missing values
78 print(f"Missing values per column:")
79 missing_summary = df.isnull().sum()
80 print(missing_summary[missing_summary > 0])
81
82
83 # Visualize missing values
84 plt.figure(figsize=(10, 6))
85 plt.subplot(1, 2, 1)
86 missing_summary[missing_summary > 0].plot(kind='bar', color='cora
87 plt.title('Missing Values by Column', fontweight='bold')
88 plt.ylabel('Count')
89 plt.xticks(rotation=45)
90
91 # Strategy 1: Drop rows with missing values
92 df_dropna = df.dropna()
93 print(f"\nStrategy 1 – Drop missing values:")
94 print(f"    Original: {df.shape[0]} rows")
95 print(f"    After drop: {df_dropna.shape[0]} rows")
96 print(f"    Lost: {df.shape[0] - df_dropna.shape[0]} rows ({((df.s
97
98
99 # Strategy 2: Fill missing values with mean/median/mode
100 df_fillna = df.copy()
101 df_fillna['income'] = df_fillna['income'].fillna(df_fillna['incom
102 df_fillna['education'] = df_fillna['education'].fillna(df_fillna[
103 print(f"\nStrategy 2 – Fill missing values:")
104 print(f"    Income: filled with median ({df['income'].median():.0f
105 print(f"    Education: filled with mode ({df['education'].mode()[0]
106
107 # Strategy 3: Use SimpleImputer
108 imputer = SimpleImputer(strategy='median')
109 df_imputed = df.copy()
110 df_imputed['income'] = imputer.fit_transform(df_imputed[['income'
111 print(f"\nStrategy 3 – SimpleImputer:")
112

```

```

113 print(f"    Used median strategy for numerical features")
114
115 # Compare strategies
116 plt.subplot(1, 2, 2)
117 strategies = ['Original', 'Drop NA', 'Fill NA', 'Imputer']
118 missing_counts = [df.isnull().sum().sum(), df_dropna.isnull().sum()
119                  df_fillna.isnull().sum().sum(), df_imputed.isnull().sum().sum()]
120 plt.bar(strategies, missing_counts, color=['red', 'orange', 'green'])
121 plt.title('Missing Values After Each Strategy', fontweight='bold')
122 plt.ylabel('Total Missing Values')
123 plt.xticks(rotation=45)
124
125 plt.tight_layout()
126 plt.show()
127
128 print(f"\nMissing values handled successfully!")
129
130
131 # 2. Handling Categorical Variables
132 print("\n=== HANDLING CATEGORICAL VARIABLES ===")
133
134 # Label Encoding
135 le = LabelEncoder()
136 df_encoded = df_fillna.copy()
137 df_encoded['education_encoded'] = le.fit_transform(df_encoded['education'])
138 df_encoded['city_encoded'] = le.fit_transform(df_encoded['city'])
139 df_encoded['department_encoded'] = le.fit_transform(df_encoded['department'])
140 df_encoded['satisfaction_encoded'] = le.fit_transform(df_encoded['satisfaction'])
141
142 print(f"\nLabel encoded values:")
143 print(f"Education: {dict(zip(le.classes_, le.transform(le.classes_)))}")
144 print(f"City: {dict(zip(le.classes_, le.transform(le.classes_)))}")
145
146 # One-Hot Encoding
147 df_onehot = df_fillna.copy()
148 education_dummies = pd.get_dummies(df_onehot['education'], prefix='education')
149 city_dummies = pd.get_dummies(df_onehot['city'], prefix='city')
150 department_dummies = pd.get_dummies(df_onehot['department'], prefix='department')
151 satisfaction_dummies = pd.get_dummies(df_onehot['satisfaction'], prefix='satisfaction')
152
153 df_onehot = pd.concat([df_onehot, education_dummies, city_dummies, department_dummies, satisfaction_dummies])
154 print(f"\nAfter one-hot encoding: {df_onehot.shape}")
155
156 # 3. Feature Scaling
157 print("\n=== FEATURE SCALING ===")
158
159 # Select numerical features for scaling
160 numerical_features = ['age', 'income', 'experience', 'performance']
161 X_numerical = df_fillna[numerical_features]
162
163 # Standardization (mean=0, std=1)
164 scaler_standard = StandardScaler()
165 X_standardized = scaler_standard.fit_transform(X_numerical)
166 print(f"\nStandardized features shape: {X_standardized.shape}")
167 print(f"Mean: {X_standardized.mean(axis=0)}")
168 print(f"Std: {X_standardized.std(axis=0)}")
169
170 # Min-Max Scaling (0-1 range)
171 scaler_minmax = MinMaxScaler()
172 X_minmax = scaler_minmax.fit_transform(X_numerical)
173 print(f"\nMin-Max scaled features shape: {X_minmax.shape}")

```



```

177 print(f"Min: {X_minmax.min(axis=0)}")
178 print(f"Max: {X_minmax.max(axis=0)}")
179
180 # 4. Feature Engineering
181 print("\n=== FEATURE ENGINEERING ===")
182
183 # Create new features
184 df_features = df_fillna.copy()
185
186 # Age groups
187 df_features['age_group'] = pd.cut(df_features['age'],
188                                   bins=[0, 25, 35, 45, 100],
189                                   labels=['Young', 'Middle', 'Seni
190
191
192 # Income categories
193 df_features['income_category'] = pd.cut(df_features['income'],
194                                         bins=[0, 30000, 50000, 700
195                                         labels=['Low', 'Medium', '
196
197 # Experience level
198 df_features['experience_level'] = pd.cut(df_features['experience'
199                                         bins=[0, 5, 10, 20, 100],
200                                         labels=['Junior', 'Mid',
201
202
203 # Performance categories
204 df_features['performance_category'] = pd.cut(df_features['perform
205                                         bins=[0, 60, 80, 90,
206                                         labels=['Poor', 'Good
207
208 # Salary ratio
209 df_features['salary_ratio'] = df_features['salary'] / df_features
210
211 # Experience to age ratio
212 df_features['experience_age_ratio'] = df_features['experience'] /
213
214 print(f"\nNew features created:")
215 print(df_features[['age_group', 'income_category', 'experience_le
216
217 # 5. Feature Selection
218 print("\n=== FEATURE SELECTION ===")
219
220 # Prepare features and target
221 X = df_features[['age', 'income', 'experience', 'performance_scor
222 y = df_features['satisfaction']
223
224
225 # Select top 2 features
226 selector = SelectKBest(score_func=f_classif, k=2)
227 X_selected = selector.fit_transform(X, y)
228
229 print(f"\nSelected features: {X.columns[selector.get_support()]}")
230 print(f"Feature scores: {selector.scores_}")
231 print(f"Selected features shape: {X_selected.shape}")
232
233 # 6. Data Splitting
234 print("\n=== DATA SPLITTING ===")
235
236 # Split the data
237 X_train, X_test, y_train, y_test = train_test_split(X, y, test_si
238
239
240 print(f"Training set size: {X_train.shape[0]}")

```

```

241 print(f"Test set size: {X_test.shape[0]}")
242 print(f"Training set target distribution:")
243 print(y_train.value_counts(normalize=True))
244 print(f"\nTest set target distribution:")
245 print(y_test.value_counts(normalize=True))
246
247 # 7. Complete Preprocessing Pipeline
248 print("\n=== COMPLETE PREPROCESSING PIPELINE ===")
249
250 # Create a complete preprocessing pipeline
251 from sklearn.pipeline import Pipeline
252 from sklearn.compose import ColumnTransformer
253
254 # Define numerical and categorical features
255 numerical_features = ['age', 'income', 'experience', 'performance']
256 categorical_features = ['education', 'city', 'department']
257
258 # Create preprocessing pipelines
259 numerical_pipeline = Pipeline([
260     ('imputer', SimpleImputer(strategy='median')),
261     ('scaler', StandardScaler())
262 ])
263
264 categorical_pipeline = Pipeline([
265     ('imputer', SimpleImputer(strategy='most_frequent')),
266     ('onehot', OneHotEncoder(handle_unknown='ignore'))
267 ])
268
269 # Combine pipelines
270
271 preprocessor = ColumnTransformer([
272     ('num', numerical_pipeline, numerical_features),
273     ('cat', categorical_pipeline, categorical_features)
274 ])
275
276 # Apply preprocessing
277 X_processed = preprocessor.fit_transform(df_fillna)
278 print(f"\nProcessed features shape: {X_processed.shape}")
279 print(f"Feature names: {preprocessor.get_feature_names_out()}")
280
281 print("\n=== PREPROCESSING SUMMARY ===")
282 print("1. Missing values handled with appropriate strategies")
283 print("2. Categorical variables encoded (label and one-hot)")
284 print("3. Features scaled (standardization and min-max)")
285 print("4. New features created through engineering")
286 print("5. Feature selection applied")
287 print("6. Data split into train/test sets")
288 print("7. Complete pipeline created for reproducibility")

```

In [...

```

1 # 2. HANDLING CATEGORICAL VARIABLES
2 print("\n2. HANDLING CATEGORICAL VARIABLES")
3 print("-" * 40)
4
5 # Work with the filled dataset
6 df_work = df_fillna.copy()
7
8 print("Categorical columns:")
9 categorical_cols = df_work.select_dtypes(include=['object']).columns
10

```

```

11 print(f"    {categorical_cols}")
12
13 # Label Encoding
14 print("\nLabel Encoding:")
15 le = LabelEncoder()
16 df_encoded = df_work.copy()
17
18 for col in ['education', 'city', 'department', 'satisfaction']:
19     df_encoded[f'{col}_encoded'] = le.fit_transform(df_encoded[col])
20     print(f"    {col}: {dict(zip(le.classes_, le.transform(le.classes_)))}")
21
22 # One-Hot Encoding
23 print("\nOne-Hot Encoding:")
24 df_onehot = df_work.copy()
25
26 # Create dummy variables
27 education_dummies = pd.get_dummies(df_onehot['education'], prefix='education')
28 city_dummies = pd.get_dummies(df_onehot['city'], prefix='city')
29 department_dummies = pd.get_dummies(df_onehot['department'], prefix='department')
30 satisfaction_dummies = pd.get_dummies(df_onehot['satisfaction'], prefix='satisfaction')
31
32 # Combine with original data
33 df_onehot = pd.concat([df_onehot, education_dummies, city_dummies, department_dummies, satisfaction_dummies], axis=1)
34
35 print(f"    Original features: {df_work.shape[1]}")
36 print(f"    After one-hot: {df_onehot.shape[1]}")
37 print(f"    Added: {df_onehot.shape[1] - df_work.shape[1]} new features")
38
39 # Visualize encoding comparison
40 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
41
42 # Original categorical distribution
43 df_work['education'].value_counts().plot(kind='bar', ax=axes[0])
44 axes[0].set_title('Original Education Distribution', fontweight='bold')
45 axes[0].set_ylabel('Count')
46 axes[0].tick_params(axis='x', rotation=45)
47
48 # Label encoded
49 df_encoded['education_encoded'].value_counts().plot(kind='bar', ax=axes[1])
50 axes[1].set_title('Label Encoded Education', fontweight='bold')
51 axes[1].set_ylabel('Count')
52 axes[1].set_xlabel('Encoded Value')
53
54 # One-hot encoded (show first few)
55 education_cols = [col for col in df_onehot.columns if col.startswith('education')]
56 df_onehot[education_cols].sum().plot(kind='bar', ax=axes[2], color='red')
57 axes[2].set_title('One-Hot Encoded Education', fontweight='bold')
58 axes[2].set_ylabel('Count')
59 axes[2].tick_params(axis='x', rotation=45)
60
61 plt.tight_layout()
62 plt.show()
63
64 print("\nCategorical variables encoded successfully!")

```

In [...

```
1  # 3. FEATURE SCALING
2  print("\n3. FEATURE SCALING")
3  print("-" * 25)
4
5
6  # Select numerical features for scaling
7  numerical_features = ['age', 'income', 'experience', 'performance']
8  X_numerical = df_work[numerical_features]
9
10 print(f"Numerical features to scale: {numerical_features}")
11 print(f"\nOriginal statistics:")
12 print(X_numerical.describe())
13
14 # Standardization (mean=0, std=1)
15 print("\nStandardization (Z-score normalization):")
16 scaler_standard = StandardScaler()
17 X_standardized = scaler_standard.fit_transform(X_numerical)
18 X_std_df = pd.DataFrame(X_standardized, columns=numerical_features)
19
20 print(f"    Mean after standardization: {X_standardized.mean(axis=0).round(3)}")
21 print(f"    Std after standardization: {X_standardized.std(axis=0).round(3)}")
22
23
24 # Min-Max Scaling (0-1 range)
25 print("\nMin-Max Scaling (0-1 normalization):")
26 scaler_minmax = MinMaxScaler()
27 X_minmax = scaler_minmax.fit_transform(X_numerical)
28 X_mm_df = pd.DataFrame(X_minmax, columns=numerical_features)
29
30 print(f"    Min after min-max: {X_minmax.min(axis=0).round(3)}")
31 print(f"    Max after min-max: {X_minmax.max(axis=0).round(3)}")
32
33 # Visualize scaling effects
34 fig, axes = plt.subplots(2, 3, figsize=(18, 10))
35 axes = axes.flatten()
36
37 for i, feature in enumerate(numerical_features):
38     axes[i].hist(X_numerical[feature], alpha=0.7, label='Original')
39     axes[i].hist(X_std_df[feature], alpha=0.7, label='Standardized')
40     axes[i].hist(X_mm_df[feature], alpha=0.7, label='Min-Max', bi
41     axes[i].set_title(f'{feature} Scaling Comparison', fontweight
42     axes[i].legend()
43     axes[i].grid(True, alpha=0.3)
44
45 plt.tight_layout()
46 plt.show()
47
48 print("\nFeature scaling completed successfully!")
```

In [...

```
1  # 4. FEATURE ENGINEERING
2  print("\n4. FEATURE ENGINEERING")
3  print("-" * 30)
4
5
6  # Create new features
7  df_features = df_work.copy()
8
9  print("Creating new features from existing ones...")
```

```

10
11 # Age groups
12 df_features['age_group'] = pd.cut(df_features['age'],
13                                   bins=[0, 25, 35, 45, 100],
14                                   labels=['Young', 'Middle', 'Seni
15
16 # Income categories
17 df_features['income_category'] = pd.cut(df_features['income'],
18                                         bins=[0, 30000, 50000, 700
19                                         labels=['Low', 'Medium', '
20
21 # Experience level
22 df_features['experience_level'] = pd.cut(df_features['experience']
23                                         bins=[0, 5, 10, 20, 100],
24                                         labels=['Junior', 'Mid',
25
26 # Performance categories
27 df_features['performance_category'] = pd.cut(df_features['perform
28                                         bins=[0, 60, 80, 90,
29                                         labels=['Poor', 'Good
30
31 # Mathematical combinations
32 df_features['salary_ratio'] = df_features['salary'] / df_features
33 df_features['experience_age_ratio'] = df_features['experience'] /
34 df_features['performance_income_ratio'] = df_features['performanc
35
36 # Interaction features
37 df_features['age_experience_interaction'] = df_features['age'] *
38 df_features['income_performance_interaction'] = df_features['inco
39
40 # Polynomial features
41 df_features['age_squared'] = df_features['age'] ** 2
42 df_features['income_squared'] = df_features['income'] ** 2
43
44 print(f"\nCreated {len(df_features.columns) - len(df_work.column
45 print("\nNew features:")
46 new_features = [col for col in df_features.columns if col not in
47 for i, feature in enumerate(new_features, 1):
48     print(f"    {i:2d}. {feature}")
49
50 # Visualize some new features
51 fig, axes = plt.subplots(2, 3, figsize=(18, 10))
52 axes = axes.flatten()
53
54 # Age groups distribution
55 df_features['age_group'].value_counts().plot(kind='bar', ax=axes[
56 axes[0].set_title('Age Groups Distribution', fontweight='bold')
57 axes[0].tick_params(axis='x', rotation=45)
58
59 # Income categories
60 df_features['income_category'].value_counts().plot(kind='bar', ax
61 axes[1].set_title('Income Categories', fontweight='bold')
62 axes[1].tick_params(axis='x', rotation=45)
63
64 # Experience levels
65 df_features['experience_level'].value_counts().plot(kind='bar', a
66 axes[2].set_title('Experience Levels', fontweight='bold')
67 axes[2].tick_params(axis='x', rotation=45)
68
69 # Salary ratio distribution

```

```

74 axes[3].hist(df_features['salary_ratio'], bins=30, color='purple')
75 axes[3].set_title('Salary Ratio Distribution', fontweight='bold')
76 axes[3].set_xlabel('Salary/Income Ratio')
77
78 # Experience-Age ratio
79 axes[4].hist(df_features['experience_age_ratio'], bins=30, color=
80 axes[4].set_title('Experience/Age Ratio', fontweight='bold')
81 axes[4].set_xlabel('Experience/Age')
82
83 # Performance categories
84 df_features['performance_category'].value_counts().plot(kind='bar
85 axes[5].set_title('Performance Categories', fontweight='bold')
86 axes[5].tick_params(axis='x', rotation=45)
87
plt.tight_layout()
plt.show()

print("\nFeature engineering completed successfully!")

```

4. Train-Test Split and Cross-Validation

In [...

```

1 # =====
2 # 4. TRAIN-TEST SPLIT AND CROSS-VALIDATION
3 # =====
4
5
6 print("TRAIN-TEST SPLIT AND CROSS-VALIDATION")
7 print("=" * 45)
8
9 print("\nWHY SPLIT DATA?")
10 print("• Prevent overfitting")
11 print("• Get unbiased model performance estimate")
12 print("• Simulate real-world performance")
13 print("• Compare different models fairly")
14
15 # Prepare data for splitting
16 numerical_features = ['age', 'income', 'experience', 'performance
17 X = df_features[numerical_features]
18 y = df_features['satisfaction']
19
20
21 print(f"\nDataset for splitting:")
22 print(f"  Features shape: {X.shape}")
23 print(f"  Target shape: {y.shape}")
24 print(f"  Target distribution:")
25 print(y.value_counts(normalize=True).round(3))
26
27 # 1. BASIC TRAIN-TEST SPLIT
28 print("\n1. BASIC TRAIN-TEST SPLIT")
29 print("-" * 35)
30
31 X_train, X_test, y_train, y_test = train_test_split(X, y, test_si
32
33 print(f"  Training set: {X_train.shape[0]} samples ({X_train.sha
34 print(f"  Test set: {X_test.shape[0]} samples ({X_test.shape[0]/
35
36
37 print(f"\n  Training target distribution:")
38 print(y_train.value_counts(normalize=True).round(3))

```

```

38 print(f"\n Test target distribution:")
39 print(y_test.value_counts(normalize=True).round(3))
40
41 # 2. CROSS-VALIDATION
42 print("\n2. CROSS-VALIDATION")
43 print("-" * 25)
44
45 # K-Fold Cross-Validation
46 print("\nK-Fold Cross-Validation:")
47 kfold = KFold(n_splits=5, shuffle=True, random_state=42)
48
49 # Create a simple model for demonstration
50 from sklearn.linear_model import LogisticRegression
51 model = LogisticRegression(random_state=42)
52
53 # Perform cross-validation
54 cv_scores = cross_val_score(model, X_train, y_train, cv=kfold, sc
55
56
57 print(f" CV Scores: {cv_scores.round(3)}")
58 print(f" Mean CV Score: {cv_scores.mean():.3f} (+/- {cv_scores.
59
60 # Stratified K-Fold (better for imbalanced datasets)
61 print("\nStratified K-Fold Cross-Validation:")
62 skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=4
63 cv_scores_stratified = cross_val_score(model, X_train, y_train, c
64
65
66 print(f" Stratified CV Scores: {cv_scores_stratified.round(3)}")
67 print(f" Mean Stratified CV Score: {cv_scores_stratified.mean()
68
69 # Visualize cross-validation results
70 fig, axes = plt.subplots(1, 2, figsize=(15, 6))
71
72 # K-Fold scores
73 axes[0].bar(range(1, 6), cv_scores, color='skyblue', alpha=0.7)
74 axes[0].axhline(y=cv_scores.mean(), color='red', linestyle='--',
75 axes[0].set_title('K-Fold Cross-Validation Scores', fontweight='b
76 axes[0].set_xlabel('Fold')
77 axes[0].set_ylabel('Accuracy')
78 axes[0].legend()
79 axes[0].grid(True, alpha=0.3)
80
81 # Stratified K-Fold scores
82 axes[1].bar(range(1, 6), cv_scores_stratified, color='lightgreen'
83 axes[1].axhline(y=cv_scores_stratified.mean(), color='red', lines
84 label=f'Mean: {cv_scores_stratified.mean():.3f}')
85 axes[1].set_title('Stratified K-Fold Cross-Validation Scores', fo
86 axes[1].set_xlabel('Fold')
87 axes[1].set_ylabel('Accuracy')
88 axes[1].legend()
89 axes[1].grid(True, alpha=0.3)
90
91
92 plt.tight_layout()
93 plt.show()
94
95 # 3. VALIDATION SET (TRAIN-VALIDATION-TEST)
96 print("\n3. TRAIN-VALIDATION-TEST SPLIT")
97 print("-" * 40)
98
99 # First split: train+val vs test
100 X_temp, X_test_final, y_temp, y_test_final = train_test_split(X,
101

```



```

102
103 # Second split: train vs validation
104 X_train_final, X_val, y_train_final, y_val = train_test_split(X_t
105
106 print(f"    Training set: {X_train_final.shape[0]} samples")
107 print(f"    Validation set: {X_val.shape[0]} samples")
108 print(f"    Test set: {X_test_final.shape[0]} samples")
109
110 print(f"\n    Training: {X_train_final.shape[0]/len(X)*100:.1f}%")
111 print(f"    Validation: {X_val.shape[0]/len(X)*100:.1f}%")
112 print(f"    Test: {X_test_final.shape[0]/len(X)*100:.1f}%")
113
114
115 # 4. TIME SERIES SPLIT (for temporal data)
116 print("\n4. TIME SERIES SPLIT")
117 print("-" * 25)
118
119 from sklearn.model_selection import TimeSeriesSplit
120
121 # Create time series data
122 np.random.seed(42)
123 time_data = np.random.randn(100, 3)
124 time_target = np.random.randn(100)
125
126 tscv = TimeSeriesSplit(n_splits=5)
127 print("    Time series splits:")
128 for i, (train_idx, val_idx) in enumerate(tscv.split(time_data)):
129     print(f"    Split {i+1}: Train {len(train_idx)} samples, Valid
130
131
132 print("\nData splitting strategies completed!")
133 print("\nKey Takeaways:")
134 print("• Always split data before any preprocessing")
135 print("• Use stratified split for imbalanced datasets")
136 print("• Cross-validation gives more robust performance estimates")
137 print("• Keep test set completely separate until final evaluation

```

5. Introduction to scikit-learn

In [...

```

1
2 # =====
3 # 5. INTRODUCTION TO SCIKIT-LEARN
4 # =====
5
6 print("INTRODUCTION TO SCIKIT-LEARN")
7 print("=" * 40)
8
9 print("\nWHAT IS SCIKIT-LEARN?")
10 print("• Most popular Python ML library")
11 print("• Built on NumPy, SciPy, and Matplotlib")
12 print("• Consistent API across all algorithms")
13 print("• Extensive documentation and community")
14 print("• Production-ready and well-tested")
15
16 print("\nKEY COMPONENTS:")
17 print("• sklearn.datasets: Sample datasets")
18 print("• sklearn.preprocessing: Data preprocessing")
19 print("• sklearn.model_selection: Model selection and validation")
20 print("• sklearn.linear_model: Linear models")

```



```

21 print("• sklearn.tree: Decision trees")
22 print("• sklearn.ensemble: Ensemble methods")
23 print("• sklearn.metrics: Model evaluation")
24 print("• sklearn.pipeline: Pipeline construction")
25
26 # 1. DATASETS
27 print("\n1. WORKING WITH DATASETS")
28 print("-" * 35)
29
30 # Load built-in datasets
31 from sklearn.datasets import load_iris, load_wine, load_breast_ca
32
33 # Load iris dataset
34 iris = load_iris()
35 print(f"\nIris Dataset:")
36 print(f"    Features: {iris.data.shape[1]}")
37 print(f"    Samples: {iris.data.shape[0]}")
38 print(f"    Classes: {len(iris.target_names)}")
39 print(f"    Target names: {iris.target_names}")
40
41 # Load wine dataset
42 wine = load_wine()
43 print(f"\nWine Dataset:")
44 print(f"    Features: {wine.data.shape[1]}")
45 print(f"    Samples: {wine.data.shape[0]}")
46 print(f"    Classes: {len(wine.target_names)}")
47
48 # Create synthetic datasets
49 print(f"\nSynthetic Datasets:")
50 X_class, y_class = make_classification(n_samples=100, n_features=
51 X_reg, y_reg = make_regression(n_samples=100, n_features=3, noise
52
53 print(f"    Classification: {X_class.shape[0]} samples, {X_class.s
54 print(f"    Regression: {X_reg.shape[0]} samples, {X_reg.shape[1]}
55
56 # 2. CONSISTENT API
57 print("\n2. CONSISTENT API PATTERN")
58 print("-" * 35)
59
60 print("\nStandard ML Workflow:")
61 print("1. Create model instance")
62 print("2. Fit model to training data")
63 print("3. Make predictions on test data")
64 print("4. Evaluate model performance")
65
66 # Example with different algorithms
67 from sklearn.linear_model import LinearRegression
68 from sklearn.tree import DecisionTreeClassifier
69 from sklearn.ensemble import RandomForestClassifier
70
71 print("\nExample with different algorithms:")
72
73 # Linear Regression
74 print("\nLinear Regression:")
75 lr = LinearRegression()
76 lr.fit(X_reg, y_reg)
77 lr_pred = lr.predict(X_reg[:5])
78 print(f"    Predictions: {lr_pred.round(2)}")
79
80 # Decision Tree

```

```

85 print("\nDecision Tree:")
86 dt = DecisionTreeClassifier(random_state=42)
87 dt.fit(X_class, y_class)
88 dt_pred = dt.predict(X_class[:5])
89 print(f" Predictions: {dt_pred}")
90
91 # Random Forest
92 print("\nRandom Forest:")
93 rf = RandomForestClassifier(random_state=42)
94 rf.fit(X_class, y_class)
95 rf_pred = rf.predict(X_class[:5])
96 print(f" Predictions: {rf_pred}")
97
98 # 3. PIPELINES
99 print("\n3. PIPELINES")
100 print("-" * 20)
101
102 print("\nPipeline Benefits:")
103 print("• Chain multiple preprocessing steps")
104 print("• Prevent data leakage")
105 print("• Make code more readable")
106 print("• Easy to deploy and reproduce")
107
108 # Create a preprocessing pipeline
109 from sklearn.pipeline import Pipeline
110 from sklearn.preprocessing import StandardScaler
111
112 # Simple pipeline
113 pipe = Pipeline([
114     ('scaler', StandardScaler()),
115     ('classifier', LogisticRegression(random_state=42))
116 ])
117
118 print("\nPipeline created:")
119 print(pipe)
120
121 # Fit and predict with pipeline
122 pipe.fit(X_class, y_class)
123 pipe_pred = pipe.predict(X_class[:5])
124 print(f"\n Pipeline predictions: {pipe_pred}")
125
126 # 4. MODEL PERSISTENCE
127 print("\n4. MODEL PERSISTENCE")
128 print("-" * 30)
129
130 import joblib
131
132 # Save model
133 joblib.dump(pipe, 'ml_pipeline.pkl')
134 print(" Model saved to 'ml_pipeline.pkl'")
135
136 # Load model
137 loaded_pipe = joblib.load('ml_pipeline.pkl')
138 print(" Model loaded from file")
139
140 # Verify loaded model works
141 loaded_pred = loaded_pipe.predict(X_class[:5])
142 print(f" Loaded model predictions: {loaded_pred}")
143
144 # 5. VISUALIZATION

```

```

149 print("\n5. SCIKIT-LEARN VISUALIZATION")
150 print("-" * 40)
151
152 # Plot decision boundary for 2D data
153 from sklearn.datasets import make_classification
154 from sklearn.linear_model import LogisticRegression
155 from sklearn.inspection import DecisionBoundaryDisplay
156
157 # Create 2D classification data
158 X_2d, y_2d = make_classification(n_samples=200, n_features=2, n_r
159                                n_informative=2, n_clusters_per_c
160
161
162 # Train logistic regression
163 lr_2d = LogisticRegression(random_state=42)
164 lr_2d.fit(X_2d, y_2d)
165
166 # Plot decision boundary
167 fig, ax = plt.subplots(1, 1, figsize=(10, 8))
168 DecisionBoundaryDisplay.from_estimator(lr_2d, X_2d, ax=ax, alpha=
169 ax.scatter(X_2d[:, 0], X_2d[:, 1], c=y_2d, cmap='viridis', edgeco
170 ax.set_title('Logistic Regression Decision Boundary', fontsize=14
171 ax.set_xlabel('Feature 1')
172 ax.set_ylabel('Feature 2')
173 plt.show()
174
175 print("\nScikit-learn introduction completed!")
176 print("\nKey Takeaways:")
177 print("• Consistent API makes learning easier")
178 print("• Pipelines prevent data leakage")
179 print("• Built-in datasets for practice")
180 print("• Excellent documentation and community support")
181 print("• Production-ready and well-tested")

```

6. Linear Regression

In [...

```

1 # =====
2 # 6. LINEAR REGRESSION
3 # =====
4
5
6 print("LINEAR REGRESSION")
7 print("=" * 25)
8
9 print("\nWHAT IS LINEAR REGRESSION?")
10 print("• Predicts continuous numerical values")
11 print("• Finds best linear relationship between features and targ
12 print("• Assumes linear relationship:  $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots$ ")
13 print("• Minimizes sum of squared errors")
14
15 print("\nWHEN TO USE:")
16 print("• Target variable is continuous")
17 print("• Linear relationship exists")
18 print("• Features are independent")
19 print("• No multicollinearity")
20
21 # 1. SIMPLE LINEAR REGRESSION
22 print("\n1. SIMPLE LINEAR REGRESSION")

```

```

23 print("-" * 40)
24
25 # Generate sample data
26 np.random.seed(42)
27 X_simple = np.random.randn(100, 1) * 10
28 y_simple = 2 * X_simple.flatten() + 3 + np.random.randn(100) * 2
29
30 # Split data
31 X_train_simple, X_test_simple, y_train_simple, y_test_simple = train_test_split(
32     X_simple, y_simple, test_size=0.2, random_state=42)
33
34 # Create and train model
35 lr_simple = LinearRegression()
36 lr_simple.fit(X_train_simple, y_train_simple)
37
38 # Make predictions
39 y_pred_simple = lr_simple.predict(X_test_simple)
40
41 # Calculate metrics
42 mse_simple = mean_squared_error(y_test_simple, y_pred_simple)
43 r2_simple = r2_score(y_test_simple, y_pred_simple)
44 mae_simple = mean_absolute_error(y_test_simple, y_pred_simple)
45
46 print(f"\nResults:")
47 print(f"    Coefficient ( $\beta_1$ ): {lr_simple.coef_[0]:.3f}")
48 print(f"    Intercept ( $\beta_0$ ): {lr_simple.intercept_:.3f}")
49 print(f"    Mean Squared Error: {mse_simple:.3f}")
50 print(f"    R-squared: {r2_simple:.3f}")
51 print(f"    Mean Absolute Error: {mae_simple:.3f}")
52
53 # Visualize simple linear regression
54 fig, axes = plt.subplots(1, 2, figsize=(15, 6))
55
56 # Training data
57 axes[0].scatter(X_train_simple, y_train_simple, color='blue', alpha=0.3)
58 axes[0].plot(X_train_simple, lr_simple.predict(X_train_simple), color='red', linewidth=2)
59 axes[0].set_title('Simple Linear Regression - Training', fontweight='bold')
60 axes[0].set_xlabel('X')
61 axes[0].set_ylabel('y')
62 axes[0].legend()
63 axes[0].grid(True, alpha=0.3)
64
65 # Test data
66 axes[1].scatter(X_test_simple, y_test_simple, color='blue', alpha=0.3)
67 axes[1].scatter(X_test_simple, y_pred_simple, color='red', alpha=0.3)
68 axes[1].plot(X_test_simple, y_pred_simple, color='red', linewidth=2)
69 axes[1].set_title('Simple Linear Regression - Testing', fontweight='bold')
70 axes[1].set_xlabel('X')
71 axes[1].set_ylabel('y')
72 axes[1].legend()
73 axes[1].grid(True, alpha=0.3)
74
75 plt.tight_layout()
76 plt.show()
77
78 # 2. MULTIPLE LINEAR REGRESSION
79 print("\n📌 2. MULTIPLE LINEAR REGRESSION")
80 print("-" * 40)
81
82 # Generate multiple features

```

```

87 X_multi, y_multi = make_regression(n_samples=200, n_features=3, n
88
89 # Split data
90 X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_
91     X_multi, y_multi, test_size=0.2, random_state=42)
92
93 # Create and train model
94 lr_multi = LinearRegression()
95 lr_multi.fit(X_train_multi, y_train_multi)
96
97 # Make predictions
98 y_pred_multi = lr_multi.predict(X_test_multi)
99
100 # Calculate metrics
101 mse_multi = mean_squared_error(y_test_multi, y_pred_multi)
102 r2_multi = r2_score(y_test_multi, y_pred_multi)
103 mae_multi = mean_absolute_error(y_test_multi, y_pred_multi)
104
105 print(f"\n📊 Results:")
106 print(f"    Coefficients: {lr_multi.coef_}")
107 print(f"    Intercept: {lr_multi.intercept_:.3f}")
108 print(f"    Mean Squared Error: {mse_multi:.3f}")
109 print(f"    R-squared: {r2_multi:.3f}")
110 print(f"    Mean Absolute Error: {mae_multi:.3f}")
111
112 # Feature importance (coefficient magnitude)
113 feature_names = [f'Feature {i+1}' for i in range(X_multi.shape[1])
114 coef_df = pd.DataFrame({
115     'Feature': feature_names,
116     'Coefficient': lr_multi.coef_,
117     'Abs_Coefficient': np.abs(lr_multi.coef_)
118 }).sort_values('Abs_Coefficient', ascending=True)
119
120 # Visualize feature importance
121 plt.figure(figsize=(10, 6))
122 plt.barh(coef_df['Feature'], coef_df['Coefficient'], color='skybl
123 plt.title('Feature Importance (Coefficients)', fontweight='bold')
124 plt.xlabel('Coefficient Value')
125 plt.grid(True, alpha=0.3)
126 plt.show()
127
128 # 3. RESIDUAL ANALYSIS
129 print("\n📊 3. RESIDUAL ANALYSIS")
130 print("-" * 30)
131
132 # Calculate residuals
133 residuals = y_test_multi - y_pred_multi
134
135 # Residual plots
136 fig, axes = plt.subplots(2, 2, figsize=(15, 10))
137
138 # Residuals vs Predicted
139 axes[0, 0].scatter(y_pred_multi, residuals, alpha=0.6)
140 axes[0, 0].axhline(y=0, color='red', linestyle='--')
141 axes[0, 0].set_title('Residuals vs Predicted', fontweight='bold')
142 axes[0, 0].set_xlabel('Predicted Values')
143 axes[0, 0].set_ylabel('Residuals')
144 axes[0, 0].grid(True, alpha=0.3)
145
146 # Residuals histogram

```

```

151 axes[0, 1].hist(residuals, bins=20, alpha=0.7, color='skyblue')
152 axes[0, 1].set_title('Residuals Distribution', fontweight='bold')
153 axes[0, 1].set_xlabel('Residuals')
154 axes[0, 1].set_ylabel('Frequency')
155 axes[0, 1].grid(True, alpha=0.3)
156
157 # Q-Q plot for normality
158 from scipy import stats
159 stats.probplot(residuals, dist="norm", plot=axes[1, 0])
160 axes[1, 0].set_title('Q-Q Plot (Normality Check)', fontweight='bo
161 axes[1, 0].grid(True, alpha=0.3)
162
163 # Actual vs Predicted
164 axes[1, 1].scatter(y_test_multi, y_pred_multi, alpha=0.6)
165 axes[1, 1].plot([y_test_multi.min(), y_test_multi.max()],
166                 [y_test_multi.min(), y_test_multi.max()], 'r--',
167 axes[1, 1].set_title('Actual vs Predicted', fontweight='bold')
168 axes[1, 1].set_xlabel('Actual Values')
169 axes[1, 1].set_ylabel('Predicted Values')
170 axes[1, 1].grid(True, alpha=0.3)
171
172 plt.tight_layout()
173 plt.show()
174
175 # 4. POLYNOMIAL REGRESSION
176 print("\n📌 4. POLYNOMIAL REGRESSION")
177 print("-" * 35)
178
179 from sklearn.preprocessing import PolynomialFeatures
180
181 # Generate non-linear data
182 X_poly = np.linspace(-3, 3, 100).reshape(-1, 1)
183 y_poly = 0.5 * X_poly.flatten()**3 - 2 * X_poly.flatten()**2 + X_
184
185 # Create polynomial features
186 poly_features = PolynomialFeatures(degree=3)
187 X_poly_features = poly_features.fit_transform(X_poly)
188
189 # Train polynomial regression
190 lr_poly = LinearRegression()
191 lr_poly.fit(X_poly_features, y_poly)
192
193 # Make predictions
194 y_pred_poly = lr_poly.predict(X_poly_features)
195
196 # Visualize polynomial regression
197 plt.figure(figsize=(12, 5))
198
199 plt.subplot(1, 2, 1)
200 plt.scatter(X_poly, y_poly, alpha=0.6, color='blue', label='Data')
201 plt.plot(X_poly, y_pred_poly, color='red', linewidth=2, label='Po
202 plt.title('Polynomial Regression (Degree 3)', fontweight='bold')
203 plt.xlabel('X')
204 plt.ylabel('y')
205 plt.legend()
206 plt.grid(True, alpha=0.3)
207
208 # Compare different degrees
209 plt.subplot(1, 2, 2)
210 degrees = [1, 2, 3, 4]

```

```

215     colors = ['blue', 'green', 'red', 'purple']
216
217     for degree, color in zip(degrees, colors):
218         poly_features_deg = PolynomialFeatures(degree=degree)
219         X_poly_deg = poly_features_deg.fit_transform(X_poly)
220         lr_deg = LinearRegression()
221         lr_deg.fit(X_poly_deg, y_poly)
222         y_pred_deg = lr_deg.predict(X_poly_deg)
223         plt.plot(X_poly, y_pred_deg, color=color, linewidth=2, label=
224
225
226     plt.scatter(X_poly, y_poly, alpha=0.3, color='black', label='Data')
227     plt.title('Polynomial Regression Comparison', fontweight='bold')
228     plt.xlabel('X')
229     plt.ylabel('y')
230     plt.legend()
231     plt.grid(True, alpha=0.3)
232
233     plt.tight_layout()
234     plt.show()
235
236     print("\n✅ Linear Regression completed!")
237     print("\n💡 Key Takeaways:")
238     print("• Linear regression finds the best linear relationship")
239     print("• Residual analysis helps validate assumptions")
240     print("• Polynomial regression can capture non-linear patterns")
241     print("• R² measures how well the model explains variance")
242     print("• Feature importance can be assessed through coefficients")

```

7. Logistic Regression

In [...

```

1
2     # =====
3     # 7. LOGISTIC REGRESSION
4     # =====
5
6     print("📊 LOGISTIC REGRESSION")
7     print("=" * 30)
8
9     print("\n🎯 WHAT IS LOGISTIC REGRESSION?")
10    print("• Classification algorithm (not regression despite the name)")
11    print("• Uses logistic function to model probability")
12    print("• Outputs probabilities between 0 and 1")
13    print("• Uses sigmoid function:  $p = 1 / (1 + e^{(-z)})$ ")
14    print("• Where  $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$ ")
15
16    print("\n📊 WHEN TO USE:")
17    print("• Binary classification problems")
18    print("• Need probability estimates")
19    print("• Linear decision boundary is appropriate")
20    print("• Interpretable coefficients are important")
21
22    # 1. BINARY LOGISTIC REGRESSION
23    print("\n📋 1. BINARY LOGISTIC REGRESSION")
24    print("-" * 40)
25
26    # Generate binary classification data
27    X_binary, y_binary = make_classification(n_samples=200, n_feature
28

```

```

29                                     n_informative=2, n_cluste
30                                     random_state=42)
31
32     # Split data
33     X_train_binary, X_test_binary, y_train_binary, y_test_binary = tr
34     X_binary, y_binary, test_size=0.2, random_state=42, stratify=
35
36     # Create and train logistic regression
37     lr_binary = LogisticRegression(random_state=42)
38     lr_binary.fit(X_train_binary, y_train_binary)
39
40
41     # Make predictions
42     y_pred_binary = lr_binary.predict(X_test_binary)
43     y_pred_proba_binary = lr_binary.predict_proba(X_test_binary)
44
45     # Calculate metrics
46     accuracy_binary = accuracy_score(y_test_binary, y_pred_binary)
47     precision_binary = precision_score(y_test_binary, y_pred_binary)
48     recall_binary = recall_score(y_test_binary, y_pred_binary)
49     f1_binary = f1_score(y_test_binary, y_pred_binary)
50
51     print(f"\n📊 Results:")
52     print(f"    Accuracy: {accuracy_binary:.3f}")
53     print(f"    Precision: {precision_binary:.3f}")
54     print(f"    Recall: {recall_binary:.3f}")
55     print(f"    F1-Score: {f1_binary:.3f}")
56     print(f"    Coefficients: {lr_binary.coef_[0]}")
57     print(f"    Intercept: {lr_binary.intercept_[0]:.3f}")
58
59
60     # Visualize decision boundary
61     fig, axes = plt.subplots(1, 2, figsize=(15, 6))
62
63     # Decision boundary
64     DecisionBoundaryDisplay.from_estimator(lr_binary, X_binary, ax=axes[0],
65     axes[0].scatter(X_binary[:, 0], X_binary[:, 1], c=y_binary, cmap=
66     axes[0].set_title('Logistic Regression Decision Boundary', fontwe
67     axes[0].set_xlabel('Feature 1')
68     axes[0].set_ylabel('Feature 2')
69
70     # Probability predictions
71     scatter = axes[1].scatter(X_test_binary[:, 0], X_test_binary[:, 1],
72     c=y_pred_proba_binary[:, 1], cmap='coolw
73     edgecolors='black', s=100)
74     axes[1].set_title('Predicted Probabilities', fontweight='bold')
75     axes[1].set_xlabel('Feature 1')
76     axes[1].set_ylabel('Feature 2')
77     plt.colorbar(scatter, ax=axes[1], label='Probability of Class 1')
78
79
80     plt.tight_layout()
81     plt.show()
82
83     # 2. MULTICLASS LOGISTIC REGRESSION
84     print("\n📊 2. MULTICLASS LOGISTIC REGRESSION")
85     print("-" * 45)
86
87     # Generate multiclass data
88     X_multi, y_multi = make_classification(n_samples=300, n_features=
89     n_informative=2, n_classes=
90     random_state=42)
91
92

```



```

93 # Split data
94 X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_
95     X_multi, y_multi, test_size=0.2, random_state=42, stratify=y_
96
97 # Create and train multiclass logistic regression
98 lr_multi = LogisticRegression(random_state=42, multi_class='ovr')
99 lr_multi.fit(X_train_multi, y_train_multi)
100
101 # Make predictions
102 y_pred_multi = lr_multi.predict(X_test_multi)
103 y_pred_proba_multi = lr_multi.predict_proba(X_test_multi)
104
105 # Calculate metrics
106 accuracy_multi = accuracy_score(y_test_multi, y_pred_multi)
107
108 print(f"\n📊 Results:")
109 print(f"    Accuracy: {accuracy_multi:.3f}")
110 print(f"    Number of classes: {len(np.unique(y_multi))}")
111 print(f"    Coefficients shape: {lr_multi.coef_.shape}")
112
113 # Classification report
114 print(f"\n📋 Classification Report:")
115 print(classification_report(y_test_multi, y_pred_multi))
116
117 # Visualize multiclass decision boundaries
118 fig, axes = plt.subplots(1, 2, figsize=(15, 6))
119
120 # Decision boundaries
121 DecisionBoundaryDisplay.from_estimator(lr_multi, X_multi, ax=axes
122 axes[0].scatter(X_multi[:, 0], X_multi[:, 1], c=y_multi, cmap='viridis')
123 axes[0].set_title('Multiclass Decision Boundaries', fontweight='bold')
124 axes[0].set_xlabel('Feature 1')
125 axes[0].set_ylabel('Feature 2')
126
127 # Confusion matrix
128 cm = confusion_matrix(y_test_multi, y_pred_multi)
129 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1])
130 axes[1].set_title('Confusion Matrix', fontweight='bold')
131 axes[1].set_xlabel('Predicted')
132 axes[1].set_ylabel('Actual')
133
134 plt.tight_layout()
135 plt.show()
136
137 # 3. ROC CURVE AND AUC
138 print("\n📈 3. ROC CURVE AND AUC")
139 print("-" * 30)
140
141 # Calculate ROC curve for binary classification
142 fpr, tpr, thresholds = roc_curve(y_test_binary, y_pred_proba_binary[:, 1])
143 auc_score = roc_auc_score(y_test_binary, y_pred_proba_binary[:, 1])
144
145 # Plot ROC curve
146 plt.figure(figsize=(10, 6))
147 plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC={auc_score:.2f})')
148 plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random')
149 plt.xlim([0.0, 1.0])
150 plt.ylim([0.0, 1.05])
151 plt.xlabel('False Positive Rate')
152 plt.ylabel('True Positive Rate')

```

```

157 plt.title('Receiver Operating Characteristic (ROC) Curve', fontwe
158 plt.legend(loc="lower right")
159 plt.grid(True, alpha=0.3)
160 plt.show()
161
162 print(f"    AUC Score: {auc_score:.3f}")
163
164 # 4. PROBABILITY CALIBRATION
165 print("\n☞ 4. PROBABILITY CALIBRATION")
166 print("-" * 35)
167
168 from sklearn.calibration import CalibratedClassifierCV, calibrati
169
170 # Create uncalibrated and calibrated classifiers
171 lr_uncalibrated = LogisticRegression(random_state=42)
172 lr_calibrated = CalibratedClassifierCV(lr_uncalibrated, method='s
173
174 # Fit both
175 lr_uncalibrated.fit(X_train_binary, y_train_binary)
176 lr_calibrated.fit(X_train_binary, y_train_binary)
177
178 # Get probabilities
179 prob_uncalibrated = lr_uncalibrated.predict_proba(X_test_binary)[
180 prob_calibrated = lr_calibrated.predict_proba(X_test_binary)[: , 1
181
182 # Plot calibration curves
183 plt.figure(figsize=(12, 5))
184
185 plt.subplot(1, 2, 1)
186 fraction_of_positives, mean_predicted_value = calibration_curve(y
187 plt.plot(mean_predicted_value, fraction_of_positives, "s-", label
188 plt.plot([0, 1], [0, 1], "k:", label="Perfectly calibrated")
189 plt.xlabel('Mean Predicted Probability')
190 plt.ylabel('Fraction of Positives')
191 plt.title('Calibration Plot - Uncalibrated', fontweight='bold')
192 plt.legend()
193 plt.grid(True, alpha=0.3)
194
195 plt.subplot(1, 2, 2)
196 fraction_of_positives, mean_predicted_value = calibration_curve(y
197 plt.plot(mean_predicted_value, fraction_of_positives, "s-", label
198 plt.plot([0, 1], [0, 1], "k:", label="Perfectly calibrated")
199 plt.xlabel('Mean Predicted Probability')
200 plt.ylabel('Fraction of Positives')
201 plt.title('Calibration Plot - Calibrated', fontweight='bold')
202 plt.legend()
203 plt.grid(True, alpha=0.3)
204
205 plt.tight_layout()
206 plt.show()
207
208 # 5. REGULARIZATION
209 print("\n☞ 5. REGULARIZATION")
210 print("-" * 25)
211
212 # Compare different regularization strengths
213 C_values = [0.01, 0.1, 1, 10, 100]
214 train_scores = []
215 test_scores = []
216
217
218
219
220

```



```

31                                     random_state=42)
32
33     # Split data
34     X_train_tree, X_test_tree, y_train_tree, y_test_tree = train_test
35         X_tree, y_tree, test_size=0.2, random_state=42, stratify=y_tr
36
37     # Create and train decision tree
38     dt_classifier = DecisionTreeClassifier(random_state=42, max_depth
39     dt_classifier.fit(X_train_tree, y_train_tree)
40
41     # Make predictions
42     y_pred_tree = dt_classifier.predict(X_test_tree)
43     y_pred_proba_tree = dt_classifier.predict_proba(X_test_tree)
44
45     # Calculate metrics
46     accuracy_tree = accuracy_score(y_test_tree, y_pred_tree)
47     precision_tree = precision_score(y_test_tree, y_pred_tree)
48     recall_tree = recall_score(y_test_tree, y_pred_tree)
49     f1_tree = f1_score(y_test_tree, y_pred_tree)
50
51     print(f"\n📊 Results:")
52     print(f"    Accuracy: {accuracy_tree:.3f}")
53     print(f"    Precision: {precision_tree:.3f}")
54     print(f"    Recall: {recall_tree:.3f}")
55     print(f"    F1-Score: {f1_tree:.3f}")
56
57     # Visualize decision tree
58     plt.figure(figsize=(15, 10))
59     plot_tree(dt_classifier, feature_names=['Feature 1', 'Feature 2']
60             class_names=['Class 0', 'Class 1'], filled=True, rounde
61     plt.title('Decision Tree Visualization', fontsize=16, fontweight=
62     plt.show()
63
64     # Visualize decision boundary
65     fig, axes = plt.subplots(1, 2, figsize=(15, 6))
66
67     # Decision boundary
68     DecisionBoundaryDisplay.from_estimator(dt_classifier, X_tree, ax=
69     axes[0].scatter(X_tree[:, 0], X_tree[:, 1], c=y_tree, cmap='virid
70     axes[0].set_title('Decision Tree Decision Boundary', fontweight=
71     axes[0].set_xlabel('Feature 1')
72     axes[0].set_ylabel('Feature 2')
73
74     # Feature importance
75     feature_importance = dt_classifier.feature_importances_
76     axes[1].bar(['Feature 1', 'Feature 2'], feature_importance, color
77     axes[1].set_title('Feature Importance', fontweight='bold')
78     axes[1].set_ylabel('Importance')
79     axes[1].grid(True, alpha=0.3)
80
81     plt.tight_layout()
82     plt.show()
83
84     # 2. DECISION TREE REGRESSOR
85     print("\n📊 2. DECISION TREE REGRESSOR")
86     print("-" * 40)
87
88     # Generate regression data
89     X_reg_tree, y_reg_tree = make_regression(n_samples=200, n_feature
90
91
92
93
94

```

```

95 # Split data
96 X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
97     X_reg_tree, y_reg_tree, test_size=0.2, random_state=42)
98
99 # Create and train decision tree regressor
100 dt_regressor = DecisionTreeRegressor(random_state=42, max_depth=3)
101 dt_regressor.fit(X_train_reg, y_train_reg)
102
103 # Make predictions
104 y_pred_reg = dt_regressor.predict(X_test_reg)
105
106 # Calculate metrics
107 mse_reg = mean_squared_error(y_test_reg, y_pred_reg)
108 r2_reg = r2_score(y_test_reg, y_pred_reg)
109 mae_reg = mean_absolute_error(y_test_reg, y_pred_reg)
110
111 print(f"\n📊 Results:")
112 print(f"    Mean Squared Error: {mse_reg:.3f}")
113 print(f"    R-squared: {r2_reg:.3f}")
114 print(f"    Mean Absolute Error: {mae_reg:.3f}")
115
116 # Visualize regression tree
117 plt.figure(figsize=(15, 10))
118 plot_tree(dt_regressor, feature_names=['Feature'], filled=True,
119           plt.title('Decision Tree Regressor Visualization', fontsize=16, fontweight='bold')
120 plt.show()
121
122 # Visualize regression results
123 plt.figure(figsize=(12, 5))
124
125 plt.subplot(1, 2, 1)
126 X_sorted = np.sort(X_test_reg, axis=0)
127 y_pred_sorted = dt_regressor.predict(X_sorted)
128 plt.scatter(X_test_reg, y_test_reg, alpha=0.6, color='blue', label='Actual')
129 plt.plot(X_sorted, y_pred_sorted, color='red', linewidth=2, label='Predicted')
130 plt.title('Decision Tree Regression', fontweight='bold')
131 plt.xlabel('Feature')
132 plt.ylabel('Target')
133 plt.legend()
134 plt.grid(True, alpha=0.3)
135
136 plt.subplot(1, 2, 2)
137 plt.scatter(y_test_reg, y_pred_reg, alpha=0.6)
138 plt.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()],
139         plt.title('Actual vs Predicted', fontweight='bold')
140 plt.xlabel('Actual Values')
141 plt.ylabel('Predicted Values')
142 plt.grid(True, alpha=0.3)
143
144 plt.tight_layout()
145 plt.show()
146
147 # 3. TREE PRUNING AND OVERFITTING
148 print("\n🔪 3. TREE PRUNING AND OVERFITTING")
149 print("-" * 40)
150
151 # Test different max_depth values
152 max_depths = range(1, 11)
153 train_scores = []
154 test_scores = []

```

```

159
160     for depth in max_depths:
161         dt_depth = DecisionTreeClassifier(random_state=42, max_depth=
162         dt_depth.fit(X_train_tree, y_train_tree)
163         train_scores.append(dt_depth.score(X_train_tree, y_train_tree)
164         test_scores.append(dt_depth.score(X_test_tree, y_test_tree))
165
166     # Plot overfitting demonstration
167     plt.figure(figsize=(12, 5))
168
169     plt.subplot(1, 2, 1)
170     plt.plot(max_depths, train_scores, 'o-', label='Training Score',
171     plt.plot(max_depths, test_scores, 'o-', label='Test Score', color
172     plt.xlabel('Max Depth')
173     plt.ylabel('Accuracy')
174     plt.title('Overfitting in Decision Trees', fontweight='bold')
175     plt.legend()
176     plt.grid(True, alpha=0.3)
177
178
179     # Show tree with different depths
180     plt.subplot(1, 2, 2)
181     dt_shallow = DecisionTreeClassifier(random_state=42, max_depth=2)
182     dt_deep = DecisionTreeClassifier(random_state=42, max_depth=10)
183     dt_shallow.fit(X_train_tree, y_train_tree)
184     dt_deep.fit(X_train_tree, y_train_tree)
185
186     # Compare decision boundaries
187     DecisionBoundaryDisplay.from_estimator(dt_shallow, X_tree, ax=plt
188     plt.scatter(X_tree[:, 0], X_tree[:, 1], c=y_tree, cmap='viridis',
189     plt.title('Shallow Tree (max_depth=2)', fontweight='bold')
190     plt.xlabel('Feature 1')
191     plt.ylabel('Feature 2')
192
193
194     plt.tight_layout()
195     plt.show()
196
197     # 4. FEATURE IMPORTANCE
198     print("\n★ 4. FEATURE IMPORTANCE")
199     print("-" * 30)
200
201     # Use a dataset with more features
202     X_multi_tree, y_multi_tree = make_classification(n_samples=200, n
203                                                     n_informative=5,
204
205
206     # Split data
207     X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_
208     X_multi_tree, y_multi_tree, test_size=0.2, random_state=42, s
209
210     # Train decision tree
211     dt_multi = DecisionTreeClassifier(random_state=42, max_depth=4)
212     dt_multi.fit(X_train_multi, y_train_multi)
213
214     # Get feature importance
215     feature_importance = dt_multi.feature_importances_
216     feature_names = [f'Feature {i+1}' for i in range(X_multi_tree.sha
217
218     # Create importance dataframe
219     importance_df = pd.DataFrame({
220         'Feature': feature_names,
221         'Importance': feature_importance
222

```

```

223     }).sort_values('Importance', ascending=True)
224
225     # Visualize feature importance
226     plt.figure(figsize=(10, 6))
227     plt.barh(importance_df['Feature'], importance_df['Importance'], color='red')
228     plt.title('Feature Importance in Decision Tree', fontweight='bold')
229     plt.xlabel('Importance')
230     plt.grid(True, alpha=0.3)
231     plt.show()
232
233     print("\n📊 Feature Importance:")
234     for i, (feature, importance) in enumerate(zip(importance_df['Feature'], importance_df['Importance'])):
235         print(f"    {feature}: {importance:.3f}")
236
237     # 5. DECISION TREE ADVANTAGES AND DISADVANTAGES
238     print("\n✅❌ 5. ADVANTAGES AND DISADVANTAGES")
239     print("-" * 45)
240
241     print("\n✅ ADVANTAGES:")
242     print("• Easy to understand and interpret")
243     print("• Can handle both numerical and categorical data")
244     print("• Requires little data preparation")
245     print("• Can handle non-linear relationships")
246     print("• Provides feature importance")
247     print("• Can handle missing values")
248
249     print("\n❌ DISADVANTAGES:")
250     print("• Prone to overfitting")
251     print("• Can be unstable (small changes in data can lead to different results)")
252     print("• Biased towards features with more levels")
253     print("• Can create biased trees if some classes dominate")
254     print("• Not suitable for continuous output prediction")
255
256     # 6. RANDOM FOREST (ENSEMBLE OF DECISION TREES)
257     print("\n🌲 6. RANDOM FOREST")
258     print("-" * 25)
259
260     # Create random forest
261     rf = RandomForestClassifier(n_estimators=100, random_state=42, max_depth=3)
262     rf.fit(X_train_multi, y_train_multi)
263
264     # Make predictions
265     y_pred_rf = rf.predict(X_test_multi)
266     accuracy_rf = accuracy_score(y_test_multi, y_pred_rf)
267
268     print(f"\n📊 Random Forest Results:")
269     print(f"    Accuracy: {accuracy_rf:.3f}")
270     print(f"    Number of trees: {rf.n_estimators}")
271
272     # Compare single tree vs random forest
273     dt_single = DecisionTreeClassifier(random_state=42, max_depth=3)
274     dt_single.fit(X_train_multi, y_train_multi)
275     accuracy_dt = accuracy_score(y_test_multi, dt_single.predict(X_test_multi))
276
277     print(f"\n📊 Comparison:")
278     print(f"    Single Decision Tree: {accuracy_dt:.3f}")
279     print(f"    Random Forest: {accuracy_rf:.3f}")
280     print(f"    Improvement: {accuracy_rf - accuracy_dt:.3f}")
281
282     print("\n✅ Decision Trees completed!")

```



```

print("\n💡 Key Takeaways:")
print("• Decision trees are interpretable and easy to understand")
print("• Prone to overfitting – use pruning and ensemble methods")
print("• Feature importance helps with feature selection")
print("• Random Forest reduces overfitting and improves performance")
print("• Good for both classification and regression tasks")

```

9. Model Evaluation Metrics

In [...

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50

# =====
# 9. MODEL EVALUATION METRICS
# =====

print("\n📊 MODEL EVALUATION METRICS")
print("=" * 35)

print("\n🎯 WHY EVALUATE MODELS?")
print("• Measure model performance")
print("• Compare different models")
print("• Identify areas for improvement")
print("• Ensure model generalizes well")
print("• Make informed decisions about deployment")

# 1. CLASSIFICATION METRICS
print("\n📊 1. CLASSIFICATION METRICS")
print("-" * 35)

# Generate classification data
X_eval, y_eval = make_classification(n_samples=1000, n_features=4,
                                   n_informative=4, n_classes=2,
                                   random_state=42)

# Split data
X_train_eval, X_test_eval, y_train_eval, y_test_eval = train_test_split(
    X_eval, y_eval, test_size=0.2, random_state=42, stratify=y_eval)

# Train multiple models for comparison
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

models = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=10),
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100),
    'SVM': SVC(random_state=42, probability=True)
}

# Train and evaluate models
results = {}
for name, model in models.items():
    model.fit(X_train_eval, y_train_eval)
    y_pred = model.predict(X_test_eval)
    y_pred_proba = model.predict_proba(X_test_eval)[:, 1] if hasattr(model, 'predict_proba') else None

    results[name] = {
        'y_pred': y_pred,
        'y_pred_proba': y_pred_proba,
    }

```



```

51         'accuracy': accuracy_score(y_test_eval, y_pred),
52         'precision': precision_score(y_test_eval, y_pred),
53         'recall': recall_score(y_test_eval, y_pred),
54         'f1': f1_score(y_test_eval, y_pred)
55     }
56
57     # Display results
58     print("\n✓ Model Comparison:")
59     print("-" * 50)
60     print(f"{'Model':<20} {'Accuracy':<10} {'Precision':<10} {'Recall':<10}")
61     print("-" * 50)
62     for name, metrics in results.items():
63         print(f"{'name':<20} {metrics['accuracy']:<10.3f} {metrics['precision']:<10.3f} {metrics['recall']:<10.3f} {metrics['f1']:<10.3f}")
64
65
66
67     # 2. CONFUSION MATRIX
68     print("\n🔍 2. CONFUSION MATRIX")
69     print("-" * 30)
70
71     # Use the best performing model
72     best_model_name = max(results.keys(), key=lambda x: results[x]['accuracy'])
73     best_model = models[best_model_name]
74     y_pred_best = results[best_model_name]['y_pred']
75
76     # Create confusion matrix
77     cm = confusion_matrix(y_test_eval, y_pred_best)
78
79     # Visualize confusion matrix
80     fig, axes = plt.subplots(1, 2, figsize=(15, 6))
81
82     # Raw confusion matrix
83     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0])
84     axes[0].set_title(f'Confusion Matrix - {best_model_name}', fontweight='bold')
85     axes[0].set_xlabel('Predicted')
86     axes[0].set_ylabel('Actual')
87
88     # Normalized confusion matrix
89     cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
90     sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', ax=axes[1])
91     axes[1].set_title(f'Normalized Confusion Matrix - {best_model_name}', fontweight='bold')
92     axes[1].set_xlabel('Predicted')
93     axes[1].set_ylabel('Actual')
94
95     plt.tight_layout()
96     plt.show()
97
98
99
100    # 3. ROC CURVE AND AUC
101    print("\n✓ 3. ROC CURVE AND AUC")
102    print("-" * 30)
103
104    # Plot ROC curves for all models
105    plt.figure(figsize=(12, 8))
106
107    for name, metrics in results.items():
108        if metrics['y_pred_proba'] is not None:
109            fpr, tpr, _ = roc_curve(y_test_eval, metrics['y_pred_proba'])
110            auc = roc_auc_score(y_test_eval, metrics['y_pred_proba'])
111            plt.plot(fpr, tpr, label=f'{name} (AUC = {auc:.3f})', linestyle='solid')
112
113    plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
114

```

```

115 plt.xlim([0.0, 1.0])
116 plt.ylim([0.0, 1.05])
117 plt.xlabel('False Positive Rate')
118 plt.ylabel('True Positive Rate')
119 plt.title('ROC Curves Comparison', fontweight='bold')
120 plt.legend()
121 plt.grid(True, alpha=0.3)
122 plt.show()
123
124 # 4. PRECISION-RECALL CURVE
125 print("\n🎯 4. PRECISION-RECALL CURVE")
126 print("-" * 35)
127
128 from sklearn.metrics import precision_recall_curve, average_precision_score
129
130 plt.figure(figsize=(12, 8))
131
132 for name, metrics in results.items():
133     if metrics['y_pred_proba'] is not None:
134         precision, recall, _ = precision_recall_curve(y_test_eval, metrics['y_pred_proba'])
135         ap = average_precision_score(y_test_eval, metrics['y_pred_proba'])
136         plt.plot(recall, precision, label=f'{name} (AP = {ap:.3f})')
137
138 # Baseline (random classifier)
139 baseline = len(y_test_eval[y_test_eval == 1]) / len(y_test_eval)
140 plt.axhline(y=baseline, color='k', linestyle='--', label=f'Random Baseline')
141
142 plt.xlim([0.0, 1.0])
143 plt.ylim([0.0, 1.05])
144 plt.xlabel('Recall')
145 plt.ylabel('Precision')
146 plt.title('Precision-Recall Curves', fontweight='bold')
147 plt.legend()
148 plt.grid(True, alpha=0.3)
149 plt.show()
150
151 # 5. REGRESSION METRICS
152 print("\n📊 5. REGRESSION METRICS")
153 print("-" * 30)
154
155 # Generate regression data
156 X_reg_eval, y_reg_eval = make_regression(n_samples=1000, n_features=10, random_state=42)
157
158 # Split data
159 X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg_eval, y_reg_eval, test_size=0.2, random_state=42)
160
161 # Train regression models
162 from sklearn.ensemble import RandomForestRegressor
163 from sklearn.svm import SVR
164
165 reg_models = {
166     'Linear Regression': LinearRegression(),
167     'Decision Tree': DecisionTreeRegressor(random_state=42, max_depth=10),
168     'Random Forest': RandomForestRegressor(random_state=42, n_estimators=100),
169     'SVR': SVR()
170 }
171
172 reg_results = {}
173 for name, model in reg_models.items():

```

```

179     model.fit(X_train_reg, y_train_reg)
180     y_pred = model.predict(X_test_reg)
181
182     reg_results[name] = {
183         'y_pred': y_pred,
184         'mse': mean_squared_error(y_test_reg, y_pred),
185         'rmse': np.sqrt(mean_squared_error(y_test_reg, y_pred)),
186         'mae': mean_absolute_error(y_test_reg, y_pred),
187         'r2': r2_score(y_test_reg, y_pred)
188     }
189
190
191     # Display regression results
192     print("\n📊 Regression Model Comparison:")
193     print("-" * 60)
194     print(f"{'Model':<20} {'MSE':<10} {'RMSE':<10} {'MAE':<10} {'R²':<10}")
195     print("-" * 60)
196     for name, metrics in reg_results.items():
197         print(f"{'name':<20} {metrics['mse']:<10.3f} {metrics['rmse']:<10.3f} {metrics['mae']:<10.3f} {metrics['r2']:<10.3f}")
198
199
200     # Visualize regression results
201     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
202     axes = axes.flatten()
203
204     for i, (name, metrics) in enumerate(reg_results.items()):
205         axes[i].scatter(y_test_reg, metrics['y_pred'], alpha=0.6)
206         axes[i].plot([y_test_reg.min(), y_test_reg.max()],
207                     [y_test_reg.min(), y_test_reg.max()], 'r--', lw=2)
208         axes[i].set_xlabel('Actual Values')
209         axes[i].set_ylabel('Predicted Values')
210         axes[i].set_title(f'{name} (R² = {metrics["r2"]:.3f})', fontweight='bold')
211         axes[i].grid(True, alpha=0.3)
212
213
214     plt.tight_layout()
215     plt.show()
216
217     # 6. CROSS-VALIDATION SCORES
218     print("\n📊 6. CROSS-VALIDATION SCORES")
219     print("-" * 35)
220
221     # Perform cross-validation on classification models
222     cv_scores = {}
223     for name, model in models.items():
224         scores = cross_val_score(model, X_eval, y_eval, cv=5, scoring='accuracy')
225         cv_scores[name] = scores
226
227
228     # Display CV results
229     print("\n📊 Cross-Validation Results:")
230     print("-" * 40)
231     for name, scores in cv_scores.items():
232         print(f"{'name':<20}: {scores.mean():.3f} (+/- {scores.std() * 2:.3f})")
233
234     # Visualize CV results
235     plt.figure(figsize=(12, 6))
236     plt.boxplot([cv_scores[name] for name in cv_scores.keys()],
237                 labels=list(cv_scores.keys()))
238     plt.title('Cross-Validation Scores Distribution', fontweight='bold')
239     plt.ylabel('Accuracy')
240     plt.xticks(rotation=45)
241     plt.grid(True, alpha=0.3)

```

```

243 plt.show()
244
245 # 7. LEARNING CURVES
246 print("\n🚧 7. LEARNING CURVES")
247 print("-" * 25)
248
249 from sklearn.model_selection import learning_curve
250
251 # Generate learning curves for the best model
252 train_sizes, train_scores, val_scores = learning_curve(
253     best_model, X_eval, y_eval, cv=5, n_jobs=-1,
254     train_sizes=np.linspace(0.1, 1.0, 10))
255
256 # Calculate mean and std
257 train_mean = np.mean(train_scores, axis=1)
258 train_std = np.std(train_scores, axis=1)
259 val_mean = np.mean(val_scores, axis=1)
260 val_std = np.std(val_scores, axis=1)
261
262 # Plot learning curves
263 plt.figure(figsize=(10, 6))
264 plt.plot(train_sizes, train_mean, 'o-', color='blue', label='Train')
265 plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, color='blue')
266 plt.plot(train_sizes, val_mean, 'o-', color='red', label='Validation')
267 plt.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, color='red')
268 plt.xlabel('Training Set Size')
plt.ylabel('Accuracy')
plt.title(f'Learning Curves - {best_model_name}', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print("\n✅ Model Evaluation completed!")
print("\n💡 Key Takeaways:")
print("• Use multiple metrics to get a complete picture")
print("• Cross-validation provides more robust estimates")
print("• Learning curves help identify bias vs variance")
print("• ROC curves are great for binary classification")
print("• Precision-Recall curves are better for imbalanced datasets")
print("• Always validate on unseen test data")

```

10. Overfitting and Underfitting

In [...

```

1 # =====
2 # 10. OVERFITTING AND UNDERFITTING
3 # =====
4
5
6 print("⚖️ OVERFITTING AND UNDERFITTING")
7 print("-" * 40)
8
9 print("\n🎯 WHAT ARE OVERFITTING AND UNDERFITTING?")
10 print("• Overfitting: Model performs well on training data but poorly on test data")
11 print("• Underfitting: Model is too simple to capture underlying patterns")
12 print("• Bias-Variance Tradeoff: Balance between model complexity and error")
13
14 print("\n📊 WHEN DO THEY OCCUR?")

```

```

15 print("• Overfitting: Model is too complex, learns noise in train
16 print("• Underfitting: Model is too simple, cannot learn patterns
17 print("• Goal: Find the sweet spot between the two")
18
19 # 1. DEMONSTRATE OVERFITTING WITH DECISION TREES
20 print("\n🌲 1. OVERFITTING WITH DECISION TREES")
21 print("-" * 45)
22
23 # Generate data
24 X_overfit, y_overfit = make_classification(n_samples=200, n_features=10,
25                                         n_informative=2, n_clusters_per_class=1,
26                                         random_state=42)
27
28
29 # Split data
30 X_train_over, X_test_over, y_train_over, y_test_over = train_test_split(
31     X_overfit, y_overfit, test_size=0.3, random_state=42, stratify=y_overfit)
32
33 # Test different tree depths
34 max_depths = range(1, 21)
35 train_scores = []
36 test_scores = []
37
38 for depth in max_depths:
39     dt = DecisionTreeClassifier(random_state=42, max_depth=depth)
40     dt.fit(X_train_over, y_train_over)
41     train_scores.append(dt.score(X_train_over, y_train_over))
42     test_scores.append(dt.score(X_test_over, y_test_over))
43
44
45 # Find optimal depth
46 optimal_depth = max_depths[np.argmax(test_scores)]
47
48 # Plot overfitting demonstration
49 plt.figure(figsize=(15, 5))
50
51 plt.subplot(1, 3, 1)
52 plt.plot(max_depths, train_scores, 'o-', label='Training Score', color='red')
53 plt.plot(max_depths, test_scores, 'o-', label='Test Score', color='blue')
54 plt.axvline(x=optimal_depth, color='green', linestyle='--', label='Optimal Depth')
55 plt.xlabel('Max Depth')
56 plt.ylabel('Accuracy')
57 plt.title('Overfitting in Decision Trees', fontweight='bold')
58 plt.legend()
59 plt.grid(True, alpha=0.3)
60
61 # Show decision boundaries for different depths
62 depths_to_show = [1, 3, 10]
63 for i, depth in enumerate(depths_to_show):
64     plt.subplot(1, 3, i+2)
65     dt_show = DecisionTreeClassifier(random_state=42, max_depth=depth)
66     dt_show.fit(X_train_over, y_train_over)
67
68     DecisionBoundaryDisplay.from_estimator(dt_show, X_overfit, ax=plt.gca())
69     plt.scatter(X_overfit[:, 0], X_overfit[:, 1], c=y_overfit, cmap=plt.cm.Rainbow)
70     plt.title(f'Max Depth = {depth}\\nTrain: {dt_show.score(X_train_over, y_train_over):.3f}'
71             fontweight='bold')
72     plt.xlabel('Feature 1')
73     plt.ylabel('Feature 2')
74
75
76 plt.tight_layout()
77 plt.show()
78

```

```

79
80 print(f"\n📊 Results:")
81 print(f"    Optimal depth: {optimal_depth}")
82 print(f"    Best training score: {max(train_scores):.3f}")
83 print(f"    Best test score: {max(test_scores):.3f}")
84 print(f"    Overfitting gap: {max(train_scores) - max(test_scores)}")
85
86 # 2. DEMONSTRATE UNDERFITTING WITH LINEAR REGRESSION
87 print("\n📉 2. UNDERFITTING WITH LINEAR REGRESSION")
88 print("-" * 50)
89
90 # Generate non-linear data
91 X_underfit = np.linspace(-3, 3, 100).reshape(-1, 1)
92 y_underfit = 0.5 * X_underfit.flatten()**3 - 2 * X_underfit.flatten()
93
94 # Split data
95 X_train_under, X_test_under, y_train_under, y_test_under = train_test_split(
96     X_underfit, y_underfit, test_size=0.3, random_state=42)
97
98 # Test different polynomial degrees
99 degrees = [1, 2, 3, 4, 5, 6]
100 train_scores_under = []
101 test_scores_under = []
102
103 for degree in degrees:
104     poly_features = PolynomialFeatures(degree=degree)
105     X_poly = poly_features.fit_transform(X_train_under)
106     X_test_poly = poly_features.transform(X_test_under)
107
108     lr = LinearRegression()
109     lr.fit(X_poly, y_train_under)
110
111     train_scores_under.append(lr.score(X_poly, y_train_under))
112     test_scores_under.append(lr.score(X_test_poly, y_test_under))
113
114 # Plot underfitting demonstration
115 plt.figure(figsize=(15, 5))
116
117 plt.subplot(1, 3, 1)
118 plt.plot(degrees, train_scores_under, 'o-', label='Training Score')
119 plt.plot(degrees, test_scores_under, 'o-', label='Test Score', color='red')
120 plt.xlabel('Polynomial Degree')
121 plt.ylabel('R² Score')
122 plt.title('Underfitting vs Overfitting', fontweight='bold')
123 plt.legend()
124 plt.grid(True, alpha=0.3)
125
126 # Show fits for different degrees
127 degrees_to_show = [1, 3, 6]
128 for i, degree in enumerate(degrees_to_show):
129     plt.subplot(1, 3, i+2)
130
131     poly_features = PolynomialFeatures(degree=degree)
132     X_poly = poly_features.fit_transform(X_underfit)
133
134     lr = LinearRegression()
135     lr.fit(X_poly, y_underfit)
136     y_pred = lr.predict(X_poly)
137
138     plt.scatter(X_underfit, y_underfit, alpha=0.6, color='blue',

```

```

143     plt.plot(X_underfit, y_pred, color='red', linewidth=2, label=
144     plt.title(f'Degree = {degree}\\nR2 = {lr.score(X_poly, y_unde
145     plt.xlabel('X')
146     plt.ylabel('y')
147     plt.legend()
148
149     plt.tight_layout()
150     plt.show()
151
152     # 3. BIAS-VARIANCE DECOMPOSITION
153     print("\\n📊 3. BIAS-VARIANCE DECOMPOSITION")
154     print("-" * 40)
155
156     # Generate data for bias-variance analysis
157     X_bv, y_bv = make_regression(n_samples=100, n_features=1, noise=1
158
159
160     # Create multiple datasets by resampling
161     n_datasets = 50
162     models = []
163     predictions = []
164
165     for i in range(n_datasets):
166         # Bootstrap sample
167         indices = np.random.choice(len(X_bv), size=len(X_bv), replace
168         X_sample = X_bv[indices]
169         y_sample = y_bv[indices]
170
171         # Train model
172         model = LinearRegression()
173         model.fit(X_sample, y_sample)
174         models.append(model)
175
176         # Make predictions on original data
177         pred = model.predict(X_bv)
178         predictions.append(pred)
179
180
181     # Calculate bias and variance
182     predictions = np.array(predictions)
183     mean_prediction = np.mean(predictions, axis=0)
184     bias_squared = np.mean((mean_prediction - y_bv) ** 2)
185     variance = np.mean(np.var(predictions, axis=0))
186     noise = np.var(y_bv - mean_prediction)
187
188     print(f"\\n📊 Bias-Variance Analysis:")
189     print(f"    Bias2: {bias_squared:.3f}")
190     print(f"    Variance: {variance:.3f}")
191     print(f"    Noise: {noise:.3f}")
192     print(f"    Total Error: {bias_squared + variance + noise:.3f}")
193
194
195     # Visualize bias-variance
196     plt.figure(figsize=(12, 5))
197
198     plt.subplot(1, 2, 1)
199     for i in range(min(10, n_datasets)):
200         plt.plot(X_bv, predictions[i], alpha=0.3, color='blue')
201     plt.plot(X_bv, mean_prediction, color='red', linewidth=3, label=
202     plt.scatter(X_bv, y_bv, color='black', alpha=0.6, label='True Dat
203     plt.title('Model Predictions (Bias-Variance)', fontweight='bold')
204     plt.xlabel('X')
205     plt.ylabel('y')
206

```



```

207 plt.legend()
208 plt.grid(True, alpha=0.3)
209
210 plt.subplot(1, 2, 2)
211 plt.bar(['Bias2', 'Variance', 'Noise'], [bias_squared, variance,
212                                             color=['red', 'blue', 'green'], alpha=0.7)
213 plt.title('Error Components', fontweight='bold')
214 plt.ylabel('Error')
215 plt.grid(True, alpha=0.3)
216
217 plt.tight_layout()
218 plt.show()
219
220
221 # 4. REGULARIZATION TO PREVENT OVERFITTING
222 print("\n🏰 4. REGULARIZATION TECHNIQUES")
223 print("-" * 40)
224
225 # Ridge Regression (L2 regularization)
226 from sklearn.linear_model import Ridge
227
228 # Test different regularization strengths
229 alphas = np.logspace(-4, 4, 50)
230 ridge_scores = []
231
232 for alpha in alphas:
233     ridge = Ridge(alpha=alpha)
234     ridge.fit(X_train_over, y_train_over)
235     ridge_scores.append(ridge.score(X_test_over, y_test_over))
236
237 # Lasso Regression (L1 regularization)
238 from sklearn.linear_model import Lasso
239
240 lasso_scores = []
241 for alpha in alphas:
242     lasso = Lasso(alpha=alpha, max_iter=1000)
243     lasso.fit(X_train_over, y_train_over)
244     lasso_scores.append(lasso.score(X_test_over, y_test_over))
245
246 # Plot regularization effects
247 plt.figure(figsize=(15, 5))
248
249 plt.subplot(1, 3, 1)
250 plt.semilogx(alphas, ridge_scores, 'b-', label='Ridge')
251 plt.semilogx(alphas, lasso_scores, 'r-', label='Lasso')
252 plt.xlabel('Regularization Strength ( $\alpha$ )')
253 plt.ylabel('R2 Score')
254 plt.title('Regularization Effect', fontweight='bold')
255 plt.legend()
256 plt.grid(True, alpha=0.3)
257
258 # Show coefficient paths
259 plt.subplot(1, 3, 2)
260 coef_paths = []
261 for alpha in alphas[::5]: # Sample every 5th alpha
262     ridge = Ridge(alpha=alpha)
263     ridge.fit(X_train_over, y_train_over)
264     coef_paths.append(ridge.coef_)
265
266 coef_paths = np.array(coef_paths)
267 for i in range(coef_paths.shape[1]):

```



```

271     plt.semilogx(alphas[::5], coef_paths[:, i], label=f'Feature {
272     plt.xlabel('Regularization Strength ( $\alpha$ )')
273     plt.ylabel('Coefficient Value')
274     plt.title('Ridge Coefficient Paths', fontweight='bold')
275     plt.legend()
276     plt.grid(True, alpha=0.3)
277
278     # Cross-validation for optimal alpha
279     from sklearn.linear_model import RidgeCV
280
281     ridge_cv = RidgeCV(alphas=alphas, cv=5)
282     ridge_cv.fit(X_train_over, y_train_over)
283
284
285     plt.subplot(1, 3, 3)
286     plt.semilogx(alphas, ridge_scores, 'b-', label='Ridge')
287     plt.axvline(x=ridge_cv.alpha_, color='red', linestyle='--', label
288     plt.xlabel('Regularization Strength ( $\alpha$ )')
289     plt.ylabel('R2 Score')
290     plt.title('Optimal Regularization', fontweight='bold')
291     plt.legend()
292     plt.grid(True, alpha=0.3)
293
294     plt.tight_layout()
295     plt.show()
296
297     print(f"\n📊 Regularization Results:")
298     print(f"    Optimal Ridge  $\alpha$ : {ridge_cv.alpha_:.3f}")
299     print(f"    Best Ridge R2: {max(ridge_scores):.3f}")
300
301     # 5. VALIDATION CURVES
302     print("\n📈 5. VALIDATION CURVES")
303     print("-" * 25)
304
305     from sklearn.model_selection import validation_curve
306
307     # Generate validation curves for different parameters
308     param_range = np.logspace(-4, 2, 20)
309     train_scores_val, test_scores_val = validation_curve(
310         Ridge(), X_overfit, y_overfit, param_name='alpha', param_rang
311         cv=5, scoring='r2', n_jobs=-1)
312
313     # Calculate mean and std
314     train_mean = np.mean(train_scores_val, axis=1)
315     train_std = np.std(train_scores_val, axis=1)
316     test_mean = np.mean(test_scores_val, axis=1)
317     test_std = np.std(test_scores_val, axis=1)
318
319     # Plot validation curves
320     plt.figure(figsize=(12, 5))
321
322     plt.subplot(1, 2, 1)
323     plt.semilogx(param_range, train_mean, 'o-', color='blue', label='
324     plt.fill_between(param_range, train_mean - train_std, train_mean
325     plt.semilogx(param_range, test_mean, 'o-', color='red', label='Va
326     plt.fill_between(param_range, test_mean - test_std, test_mean + t
327     plt.xlabel('Regularization Strength ( $\alpha$ )')
328     plt.ylabel('R2 Score')
329     plt.title('Validation Curves - Ridge Regression', fontweight='bol
330     plt.legend()
331     plt.grid(True, alpha=0.3)
332

```

```

335
336 # Learning curves for different sample sizes
337 plt.subplot(1, 2, 2)
338 train_sizes, train_scores_lc, val_scores_lc = learning_curve(
339     Ridge(alpha=0.1), X_overfit, y_overfit, cv=5, n_jobs=-1,
340     train_sizes=np.linspace(0.1, 1.0, 10))
341
342 train_mean_lc = np.mean(train_scores_lc, axis=1)
343 train_std_lc = np.std(train_scores_lc, axis=1)
344 val_mean_lc = np.mean(val_scores_lc, axis=1)
345 val_std_lc = np.std(val_scores_lc, axis=1)
346
347 plt.plot(train_sizes, train_mean_lc, 'o-', color='blue', label='T
348 plt.fill_between(train_sizes, train_mean_lc - train_std_lc, train
349 plt.plot(train_sizes, val_mean_lc, 'o-', color='red', label='Vali
350 plt.fill_between(train_sizes, val_mean_lc - val_std_lc, val_mean
351 plt.xlabel('Training Set Size')
352 plt.ylabel('R2 Score')
353 plt.title('Learning Curves - Ridge Regression', fontweight='bold'
354 plt.legend()
355 plt.grid(True, alpha=0.3)
356
357
358 plt.tight_layout()
359 plt.show()
360
361 # 6. PREVENTION STRATEGIES
362 print("\n💡 6. PREVENTION STRATEGIES")
363 print("-" * 35)
364
365 print("\n✅ PREVENTING OVERFITTING:")
366 print("• Use cross-validation")
367 print("• Apply regularization (L1, L2)")
368 print("• Reduce model complexity")
369 print("• Increase training data")
370 print("• Use ensemble methods")
371 print("• Early stopping")
372 print("• Dropout (for neural networks)")
373
374 print("\n✅ PREVENTING UNDERFITTING:")
375 print("• Increase model complexity")
376 print("• Add more features")
377 print("• Reduce regularization")
378 print("• Use more sophisticated algorithms")
379 print("• Feature engineering")
380
381 print("\n✅ GENERAL STRATEGIES:")
382 print("• Always use train/validation/test splits")
383 print("• Monitor training and validation performance")
384 print("• Use learning curves to diagnose issues")
385 print("• Try different algorithms")
386 print("• Collect more data if possible")
387
388 print("\n✅ Overfitting and Underfitting completed!")
389 print("\n💡 Key Takeaways:")
390 print("• Overfitting: High training performance, low test perform
391 print("• Underfitting: Low performance on both training and test"
392 print("• Use validation curves to find optimal parameters")
393 print("• Regularization helps prevent overfitting")

```

```
print("• Learning curves help diagnose bias vs variance")
print("• Always validate on unseen data")
```

11. Feature Selection Techniques

In [...

```
1
2 # =====
3 # 11. FEATURE SELECTION TECHNIQUES
4 # =====
5
6 print("🔍 FEATURE SELECTION TECHNIQUES")
7 print("=" * 35)
8
9 print("\n🎯 WHY SELECT FEATURES?")
10 print("• Reduce overfitting")
11 print("• Improve model performance")
12 print("• Reduce training time")
13 print("• Improve interpretability")
14 print("• Remove irrelevant or redundant features")
15
16 print("\n📊 TYPES OF FEATURE SELECTION:")
17 print("• Filter methods: Statistical tests")
18 print("• Wrapper methods: Model-based selection")
19 print("• Embedded methods: Built into model training")
20 print("• Univariate vs Multivariate")
21
22
23 # 1. UNIVARIATE FEATURE SELECTION
24 print("\n📊 1. UNIVARIATE FEATURE SELECTION")
25 print("-" * 40)
26
27 # Generate dataset with many features
28 X_fs, y_fs = make_classification(n_samples=1000, n_features=20, n
29                                n_informative=10, n_clusters_per_
30
31 # Split data
32 X_train_fs, X_test_fs, y_train_fs, y_test_fs = train_test_split(
33     X_fs, y_fs, test_size=0.2, random_state=42, stratify=y_fs)
34
35
36 print(f"\n📄 Dataset: {X_fs.shape[0]} samples, {X_fs.shape[1]} f
37
38 # SelectKBest with f_classif
39 selector_kbest = SelectKBest(score_func=f_classif, k=10)
40 X_selected_kbest = selector_kbest.fit_transform(X_train_fs, y_tra
41
42 print(f"\n📄 SelectKBest Results:")
43 print(f"    Selected features: {X_selected_kbest.shape[1]}")
44 print(f"    Feature scores: {selector_kbest.scores_}")
45 print(f"    Selected indices: {selector_kbest.get_support(indices=
46
47 # Visualize feature scores
48 plt.figure(figsize=(15, 6))
49
50
51 plt.subplot(1, 2, 1)
52 feature_scores = selector_kbest.scores_
53 feature_indices = np.argsort(feature_scores)[::-1]
54 plt.bar(range(len(feature_scores)), feature_scores[feature_indice
55 plt.title('Feature Scores (F-statistic)', fontweight='bold')
```

```

55 plt.xlabel('Feature Rank')
56 plt.ylabel('F-score')
57 plt.grid(True, alpha=0.3)
58
59 plt.subplot(1, 2, 2)
60 selected_mask = selector_kbest.get_support()
61 plt.bar(range(len(selected_mask)), selected_mask, color='lightgreen')
62 plt.title('Selected Features (Top 10)', fontweight='bold')
63 plt.xlabel('Feature Index')
64 plt.ylabel('Selected (1) or Not (0)')
65 plt.grid(True, alpha=0.3)
66
67 plt.tight_layout()
68 plt.show()
69
70
71 # 2. RECURSIVE FEATURE ELIMINATION
72 print("\n🔄 2. RECURSIVE FEATURE ELIMINATION")
73 print("-" * 45)
74
75 # RFE with Logistic Regression
76 rfe = RFE(estimator=LogisticRegression(random_state=42), n_features_to_select=10)
77 rfe.fit(X_train_fs, y_train_fs)
78
79 X_selected_rfe = rfe.transform(X_train_fs)
80
81 print(f"\n🔄 RFE Results:")
82 print(f"    Selected features: {X_selected_rfe.shape[1]}")
83 print(f"    Feature rankings: {rfe.ranking_}")
84 print(f"    Selected indices: {rfe.get_support(indices=True)}")
85
86 # Visualize RFE results
87 plt.figure(figsize=(12, 5))
88
89 plt.subplot(1, 2, 1)
90 plt.bar(range(len(rfe.ranking_)), rfe.ranking_, color='coral')
91 plt.title('Feature Rankings (RFE)', fontweight='bold')
92 plt.xlabel('Feature Index')
93 plt.ylabel('Ranking (1 = Best)')
94 plt.grid(True, alpha=0.3)
95
96 plt.subplot(1, 2, 2)
97 selected_mask_rfe = rfe.get_support()
98 plt.bar(range(len(selected_mask_rfe)), selected_mask_rfe, color='lightgreen')
99 plt.title('Selected Features (RFE)', fontweight='bold')
100 plt.xlabel('Feature Index')
101 plt.ylabel('Selected (1) or Not (0)')
102 plt.grid(True, alpha=0.3)
103
104 plt.tight_layout()
105 plt.show()
106
107
108 # 3. EMBEDDED METHODS
109 print("\n 3. EMBEDDED METHODS")
110 print("-" * 25)
111
112 # L1 Regularization (Lasso)
113 from sklearn.linear_model import LassoCV
114
115 lasso_cv = LassoCV(cv=5, random_state=42)
116 lasso_cv.fit(X_train_fs, y_train_fs)
117

```

```

119
120 # Select features with non-zero coefficients
121 lasso_mask = lasso_cv.coef_ != 0
122 X_selected_lasso = X_train_fs[:, lasso_mask]
123
124 print(f"\n📊 Lasso Results:")
125 print(f"    Selected features: {np.sum(lasso_mask)}")
126 print(f"    Optimal alpha: {lasso_cv.alpha_:.6f}")
127 print(f"    Selected indices: {np.where(lasso_mask)[0]}")
128
129 # Random Forest Feature Importance
130 rf_selector = RandomForestClassifier(n_estimators=100, random_state=42)
131 rf_selector.fit(X_train_fs, y_train_fs)
132
133 # Select features based on importance threshold
134 importance_threshold = 0.01
135 rf_mask = rf_selector.feature_importances_ > importance_threshold
136 X_selected_rf = X_train_fs[:, rf_mask]
137
138 print(f"\n🌲 Random Forest Results:")
139 print(f"    Selected features: {np.sum(rf_mask)}")
140 print(f"    Importance threshold: {importance_threshold}")
141 print(f"    Selected indices: {np.where(rf_mask)[0]}")
142
143 # Visualize embedded methods
144 plt.figure(figsize=(15, 5))
145
146 plt.subplot(1, 3, 1)
147 plt.bar(range(len(lasso_cv.coef_)), lasso_cv.coef_, color='orange')
148 plt.title('Lasso Coefficients', fontweight='bold')
149 plt.xlabel('Feature Index')
150 plt.ylabel('Coefficient Value')
151 plt.grid(True, alpha=0.3)
152
153 plt.subplot(1, 3, 2)
154 plt.bar(range(len(rf_selector.feature_importances_)), rf_selector.feature_importances_)
155 plt.axhline(y=importance_threshold, color='red', linestyle='--')
156 plt.title('Random Forest Feature Importance', fontweight='bold')
157 plt.xlabel('Feature Index')
158 plt.ylabel('Importance')
159 plt.legend()
160 plt.grid(True, alpha=0.3)
161
162 plt.subplot(1, 3, 3)
163 methods = ['SelectKBest', 'RFE', 'Lasso', 'Random Forest']
164 selected_counts = [X_selected_kbest.shape[1], X_selected_rfe.shape[1],
165                    np.sum(lasso_mask), np.sum(rf_mask)]
166 plt.bar(methods, selected_counts, color=['skyblue', 'coral', 'orange', 'lightgreen'])
167 plt.title('Features Selected by Each Method', fontweight='bold')
168 plt.ylabel('Number of Features')
169 plt.xticks(rotation=45)
170 plt.grid(True, alpha=0.3)
171
172 plt.tight_layout()
173 plt.show()
174
175 # 4. COMPARISON OF METHODS
176 print("\n📊 4. COMPARISON OF METHODS")
177 print("-" * 35)
178
179
180
181
182

```

```

183 # Train models with different feature sets
184 methods_results = {}
185
186 # Original features
187 lr_original = LogisticRegression(random_state=42)
188 lr_original.fit(X_train_fs, y_train_fs)
189 methods_results['Original'] = lr_original.score(X_test_fs, y_test_fs)
190
191 # SelectKBest
192 lr_kbest = LogisticRegression(random_state=42)
193 lr_kbest.fit(X_selected_kbest, y_train_fs)
194 X_test_kbest = selector_kbest.transform(X_test_fs)
195 methods_results['SelectKBest'] = lr_kbest.score(X_test_kbest, y_test_fs)
196
197 # RFE
198 lr_rfe = LogisticRegression(random_state=42)
199 lr_rfe.fit(X_selected_rfe, y_train_fs)
200 X_test_rfe = rfe.transform(X_test_fs)
201 methods_results['RFE'] = lr_rfe.score(X_test_rfe, y_test_fs)
202
203 # Lasso
204 lr_lasso = LogisticRegression(random_state=42)
205 lr_lasso.fit(X_selected_lasso, y_train_fs)
206 X_test_lasso = X_test_fs[:, lasso_mask]
207 methods_results['Lasso'] = lr_lasso.score(X_test_lasso, y_test_fs)
208
209 # Random Forest
210 lr_rf = LogisticRegression(random_state=42)
211 lr_rf.fit(X_selected_rf, y_train_fs)
212 X_test_rf = X_test_fs[:, rf_mask]
213 methods_results['Random Forest'] = lr_rf.score(X_test_rf, y_test_fs)
214
215 # Display results
216 print("\n📊 Performance Comparison:")
217 print("-" * 40)
218 print(f"{'Method':<15} {'Features':<10} {'Accuracy':<10}")
219 print("-" * 40)
220 print(f"{'Original':<15} {X_train_fs.shape[1]:<10} {methods_results['Original']:<10}")
221 print(f"{'SelectKBest':<15} {X_selected_kbest.shape[1]:<10} {methods_results['SelectKBest']:<10}")
222 print(f"{'RFE':<15} {X_selected_rfe.shape[1]:<10} {methods_results['RFE']:<10}")
223 print(f"{'Lasso':<15} {np.sum(lasso_mask):<10} {methods_results['Lasso']:<10}")
224 print(f"{'Random Forest':<15} {np.sum(rf_mask):<10} {methods_results['Random Forest']:<10}")
225
226 # Visualize comparison
227 plt.figure(figsize=(12, 5))
228
229 plt.subplot(1, 2, 1)
230 methods = list(methods_results.keys())
231 accuracies = list(methods_results.values())
232 plt.bar(methods, accuracies, color=['red', 'skyblue', 'coral', 'orange'])
233 plt.title('Model Performance Comparison', fontweight='bold')
234 plt.ylabel('Accuracy')
235 plt.xticks(rotation=45)
236 plt.grid(True, alpha=0.3)
237
238 plt.subplot(1, 2, 2)
239 feature_counts = [X_train_fs.shape[1], X_selected_kbest.shape[1],
240                  np.sum(lasso_mask), np.sum(rf_mask)]
241 plt.bar(methods, feature_counts, color=['red', 'skyblue', 'coral', 'orange'])
242 plt.title('Number of Features Selected', fontweight='bold')

```

```

247 plt.ylabel('Number of Features')
248 plt.xticks(rotation=45)
249 plt.grid(True, alpha=0.3)
250
251 plt.tight_layout()
252 plt.show()
253
254 # 5. FEATURE SELECTION PIPELINE
255 print("\n🔧 5. FEATURE SELECTION PIPELINE")
256 print("-" * 40)
257
258 # Create a complete pipeline
259 from sklearn.pipeline import Pipeline
260
261 # Pipeline with feature selection
262 pipeline = Pipeline([
263     ('scaler', StandardScaler()),
264     ('selector', SelectKBest(score_func=f_classif, k=10)),
265     ('classifier', LogisticRegression(random_state=42))
266 ])
267
268 # Fit pipeline
269 pipeline.fit(X_train_fs, y_train_fs)
270
271 # Evaluate pipeline
272 pipeline_score = pipeline.score(X_test_fs, y_test_fs)
273
274 print(f"\n🔧 Pipeline Results:")
275 print(f"    Pipeline accuracy: {pipeline_score:.3f}")
276 print(f"    Selected features: {pipeline.named_steps['selector'].get_support().sum()}")
277
278 # Cross-validation with pipeline
279 cv_scores_pipeline = cross_val_score(pipeline, X_fs, y_fs, cv=5,
280                                     scoring='accuracy')
281
282 print(f"\n📊 Cross-Validation Results:")
283 print(f"    Mean CV Score: {cv_scores_pipeline.mean():.3f} (+/- {cv_scores_pipeline.std():.3f})")
284
285 # 6. FEATURE SELECTION BEST PRACTICES
286 print("\n💡 6. FEATURE SELECTION BEST PRACTICES")
287 print("-" * 45)
288
289 print("\n✅ WHEN TO USE EACH METHOD:")
290 print("• Filter methods: Quick screening, large datasets")
291 print("• Wrapper methods: When you have computational resources")
292 print("• Embedded methods: When using specific algorithms")
293 print("• Univariate: When features are independent")
294 print("• Multivariate: When features interact")
295
296 print("\n✅ BEST PRACTICES:")
297 print("• Always use cross-validation")
298 print("• Consider feature interactions")
299 print("• Don't over-optimize on validation set")
300 print("• Consider domain knowledge")
301 print("• Monitor for data leakage")
302 print("• Use multiple methods and compare")
303
304 print("\n✅ COMMON PITFALLS:")
305 print("• Selecting features based on test set")
306 print("• Ignoring feature interactions")
307 print("• Overfitting to validation set")
308

```



```

311 print("• Not considering computational cost")
312 print("• Ignoring business context")
313
314 # 7. FEATURE IMPORTANCE ANALYSIS
315 print("\n🌟 7. FEATURE IMPORTANCE ANALYSIS")
316 print("-" * 40)
317
318 # Analyze feature importance from different methods
319 importance_analysis = pd.DataFrame({
320     'Feature': [f'Feature_{i}' for i in range(X_fs.shape[1])],
321     'F_Score': selector_kbest.scores_,
322     'RFE_Ranking': rfe.ranking_,
323     'Lasso_Coeff': lasso_cv.coef_,
324     'RF_Importance': rf_selector.feature_importances_
325 })
326
327 # Normalize scores for comparison
328 importance_analysis['F_Score_Norm'] = (importance_analysis['F_Score'] - importance_analysis['F_Score'].min()) / (importance_analysis['F_Score'].max() - importance_analysis['F_Score'].min())
329 importance_analysis['RF_Importance_Norm'] = importance_analysis['RF_Importance'] / importance_analysis['RF_Importance'].max()
330 importance_analysis['Lasso_Coeff_Abs'] = np.abs(importance_analysis['Lasso_Coeff'])
331
332 # Display top features
333 print("\n📊 Top 10 Features by Different Methods:")
334 print("-" * 50)
335 print(importance_analysis.nlargest(10, 'F_Score_Norm')[['Feature', 'F_Score_Norm', 'RF_Importance_Norm', 'Lasso_Coeff_Abs']])
336
337 # Visualize feature importance correlation
338 plt.figure(figsize=(12, 4))
339
340 # Subplot 1: F-Score vs RF Importance
341 plt.subplot(1, 3, 1)
342 plt.scatter(importance_analysis['F_Score_Norm'], importance_analysis['RF_Importance_Norm'])
343 plt.xlabel('F-Score (Normalized)')
344 plt.ylabel('RF Importance (Normalized)')
345 plt.title('F-Score vs RF Importance', fontweight='bold')
346 plt.grid(True, alpha=0.3)
347
348 # Subplot 2: F-Score vs Lasso Coefficients
349 plt.subplot(1, 3, 2)
350 plt.scatter(importance_analysis['F_Score_Norm'], importance_analysis['Lasso_Coeff_Abs'])
351 plt.xlabel('F-Score (Normalized)')
352 plt.ylabel('Lasso |Coefficient|')
353 plt.title('F-Score vs Lasso Coefficients', fontweight='bold')
354 plt.grid(True, alpha=0.3)
355
356 # Subplot 3: RF Importance vs Lasso Coefficients
357 plt.subplot(1, 3, 3)
358 plt.scatter(importance_analysis['RF_Importance_Norm'], importance_analysis['Lasso_Coeff_Abs'])
359 plt.xlabel('RF Importance (Normalized)')
360 plt.ylabel('Lasso |Coefficient|')
361 plt.title('RF Importance vs Lasso Coefficients', fontweight='bold')
362 plt.grid(True, alpha=0.3)
363
364 plt.tight_layout()
365 plt.show()
366
367 print("\n✅ Feature Selection completed!")
368 print("\n💡 Key Takeaways:")
369 print("• Feature selection can improve model performance and interpretability")
370 print("• Different methods work better for different scenarios")
371 print("• Always validate feature selection with cross-validation")
372 print("• Consider the trade-off between performance and interpretability")

```



```
print("• Use domain knowledge to guide feature selection")
print("• Monitor for overfitting during feature selection")
```

Summary and Next Steps

In [...

```
1 # =====
2 # SUMMARY AND NEXT STEPS
3 # =====
4
5
6 print("MODULE 8 COMPLETED!")
7 print("=" * 25)
8
9 print("\nWHAT WE COVERED:")
10 print("1. Introduction to Machine Learning")
11 print("    • Supervised, Unsupervised, and Reinforcement Learning")
12 print("    • Common ML problems and workflow")
13
14 print("\n2. Types of Machine Learning Problems")
15 print("    • Classification, Regression, Clustering, Dimensionality Reduction")
16 print("    • Visual examples and use cases")
17
18 print("\n3. Data Preprocessing and Feature Engineering")
19 print("    • Handling missing values, categorical variables")
20 print("    • Feature scaling, engineering, and selection")
21 print("    • Complete preprocessing pipelines")
22
23
24 print("\n4. Train-Test Split and Cross-Validation")
25 print("    • Data splitting strategies")
26 print("    • K-fold and stratified cross-validation")
27 print("    • Time series splits")
28
29 print("\n5. Introduction to scikit-learn")
30 print("    • Consistent API, pipelines, model persistence")
31 print("    • Built-in datasets and visualization tools")
32
33 print("\n6. Linear Regression")
34 print("    • Simple and multiple linear regression")
35 print("    • Residual analysis, polynomial regression")
36 print("    • Feature importance and interpretation")
37
38
39 print("\n7. Logistic Regression")
40 print("    • Binary and multiclass classification")
41 print("    • ROC curves, probability calibration")
42 print("    • Regularization techniques")
43
44 print("\n8. Decision Trees")
45 print("    • Classification and regression trees")
46 print("    • Overfitting, pruning, feature importance")
47 print("    • Random Forest ensemble")
48
49 print("\n9. Model Evaluation Metrics")
50 print("    • Classification and regression metrics")
51 print("    • Confusion matrices, ROC curves")
52 print("    • Cross-validation and learning curves")
53
54 print("\n10. Overfitting and Underfitting")
```

```

55     print("    • Bias-variance tradeoff")
56     print("    • Regularization techniques")
57     print("    • Validation curves and prevention strategies")
58
59     print("\n11. Feature Selection Techniques")
60     print("    • Filter, wrapper, and embedded methods")
61     print("    • Performance comparison")
62     print("    • Best practices and pitfalls")
63
64     print("\nNEXT STEPS:")
65     print("• Practice with real datasets")
66     print("• Explore ensemble methods (Random Forest, Gradient Boosti")
67     print("• Learn about neural networks and deep learning")
68     print("• Study advanced topics (SVM, Naive Bayes, Clustering)")
69     print("• Work on end-to-end ML projects")
70     print("• Learn about model deployment and MLOps")
71
72     print("\nKEY TAKEAWAYS:")
73     print("• Always split your data properly")
74     print("• Use cross-validation for robust evaluation")
75     print("• Preprocessing is crucial for model performance")
76     print("• Start simple, then add complexity")
77     print("• Visualize your data and results")
78     print("• Understand the bias-variance tradeoff")
79     print("• Feature engineering can make a big difference")
80
81     print("\nPRACTICAL EXERCISES:")
82     print("1. Apply these techniques to a real dataset")
83     print("2. Compare different algorithms on the same data")
84     print("3. Experiment with feature engineering")
85     print("4. Build a complete ML pipeline")
86     print("5. Deploy a model and monitor its performance")
87
88     print("\nRECOMMENDED RESOURCES:")
89     print("• Scikit-learn documentation: https://scikit-learn.org/")
90     print("• Kaggle Learn: https://www.kaggle.com/learn")
91     print("• Hands-On Machine Learning by Aurélien Géron")
92     print("• The Elements of Statistical Learning by Hastie, Tibshira")
93
94     print("\nCONGRATULATIONS!")
95     print("You've completed a comprehensive introduction to Machine L")
96     print("Keep practicing and building projects to solidify your und")
97     print("\nHappy Learning!")

```